Transformers for NLP & Large Language Models

Intro to AI - Spring 2025 Class

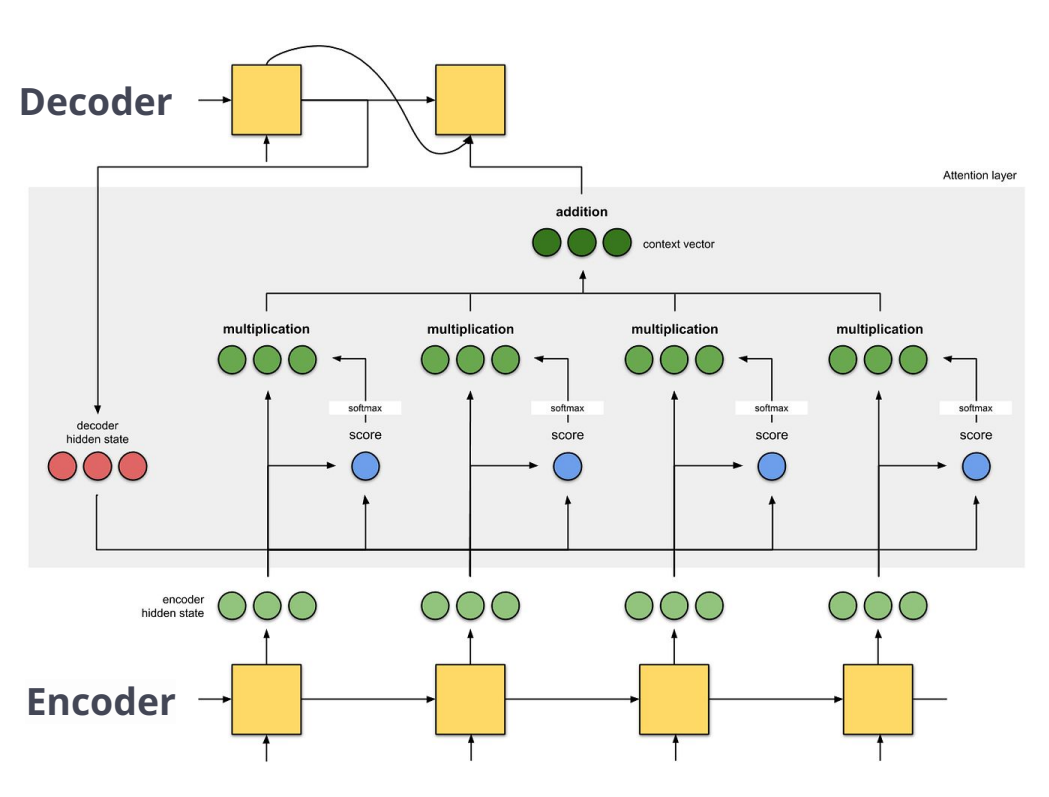
Eleni Metheniti - eleni.metheniti@univ-tlse3.fr

ANITI

Table of contents

1. Self-attention

Were you paying *attention* last week?



Self-attention

Traditional attention mechanisms are good… but dependent to current model!

What if attention were independent…

Self-attention: type of attention in a Transformer NN (Vaswani et al., 2017)

Instead of encoder-decoder hidden states, we compare `Keys:Values - Queries`

Key:Value - Query

Inspiration from Information Retrieval

Key	Value
Marchand	Léon
Riner	Teddy
Lebrun	Félix
Laurin	Althéa
Brunet	Manon

QUERY

“Brunet”

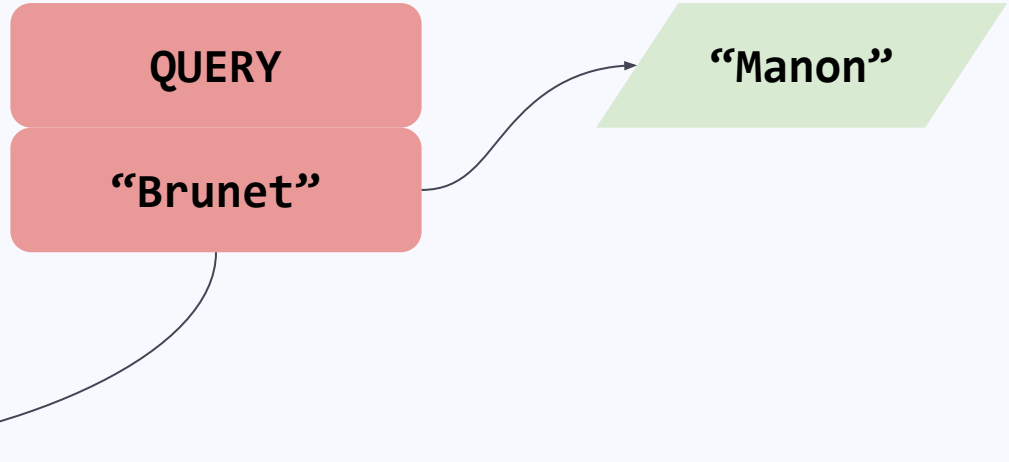
https://d2l.ai/chapter_attention-mechanisms-and-transformers/queries-keys-values.html

Key:Value - Query

Inspiration from Information Retrieval

Key	Value
Marchand	Léon
Riner	Teddy
Lebrun	Félix
Laurin	Althéa
Brunet	Manon

There is an exact match in the database!



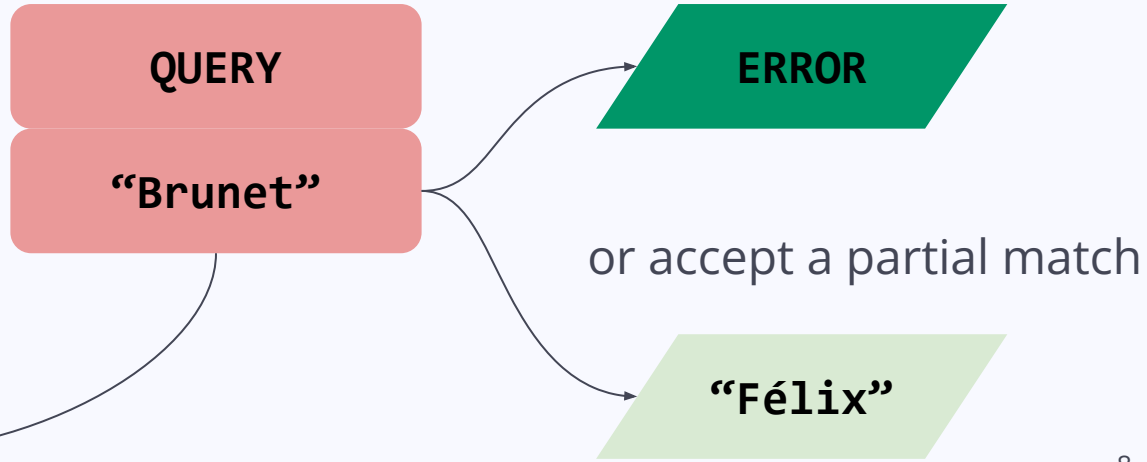
https://d2l.ai/chapter_attention-mechanisms-and-transformers/queries-keys-values.html

Key:Value - Query

Inspiration from Information Retrieval

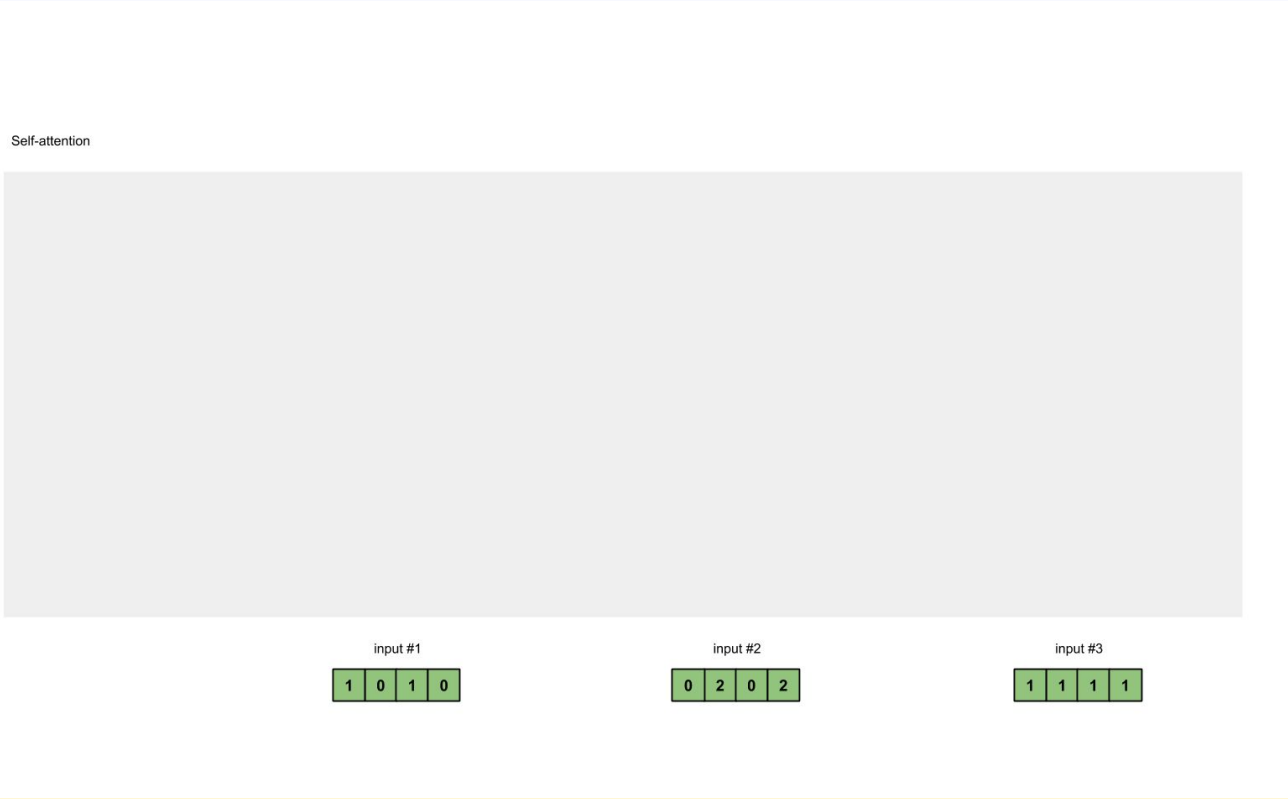
Key	Value
Marchand	Léon
Riner	Teddy
Lebrun	Félix
Laurin	Althéa
Vaast	Kauli

There is no exact match in the database!



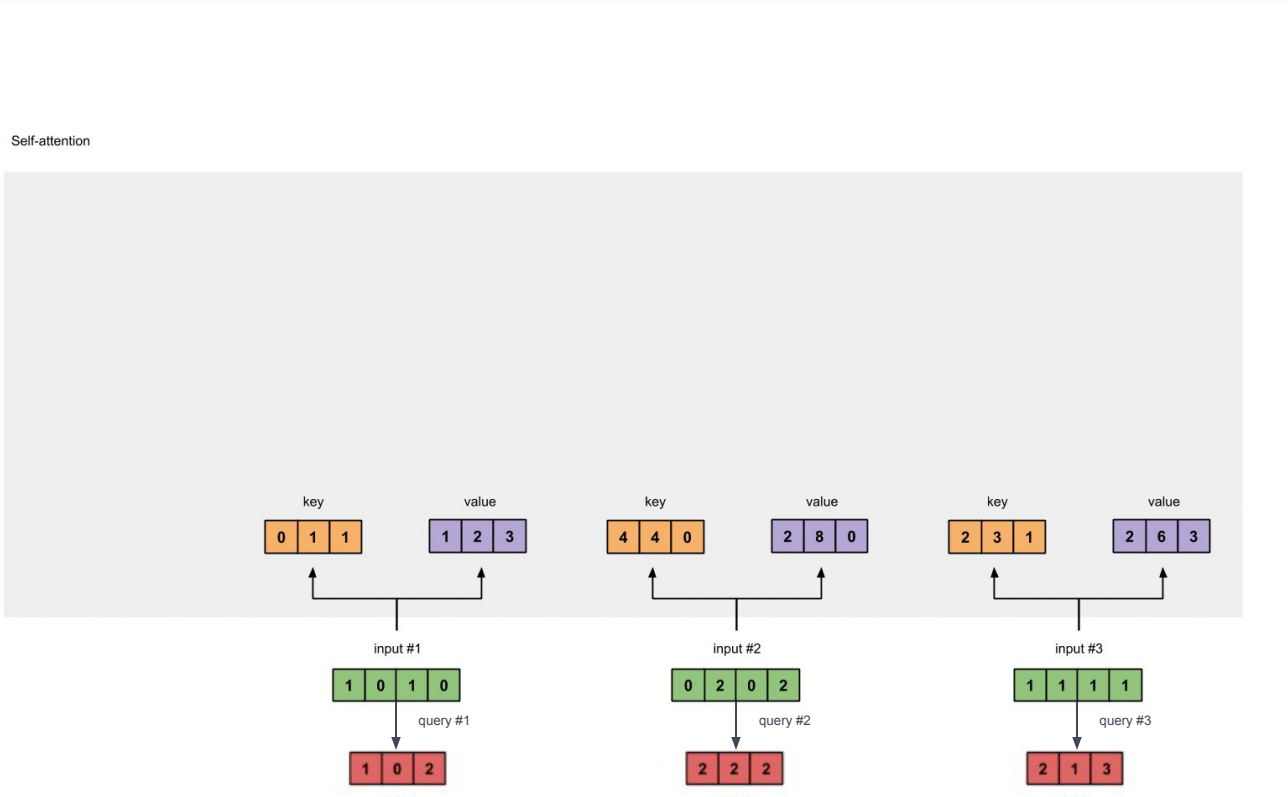
https://d2l.ai/chapter_attention-mechanisms-and-transformers/queries-keys-values.html

Self-attention steps



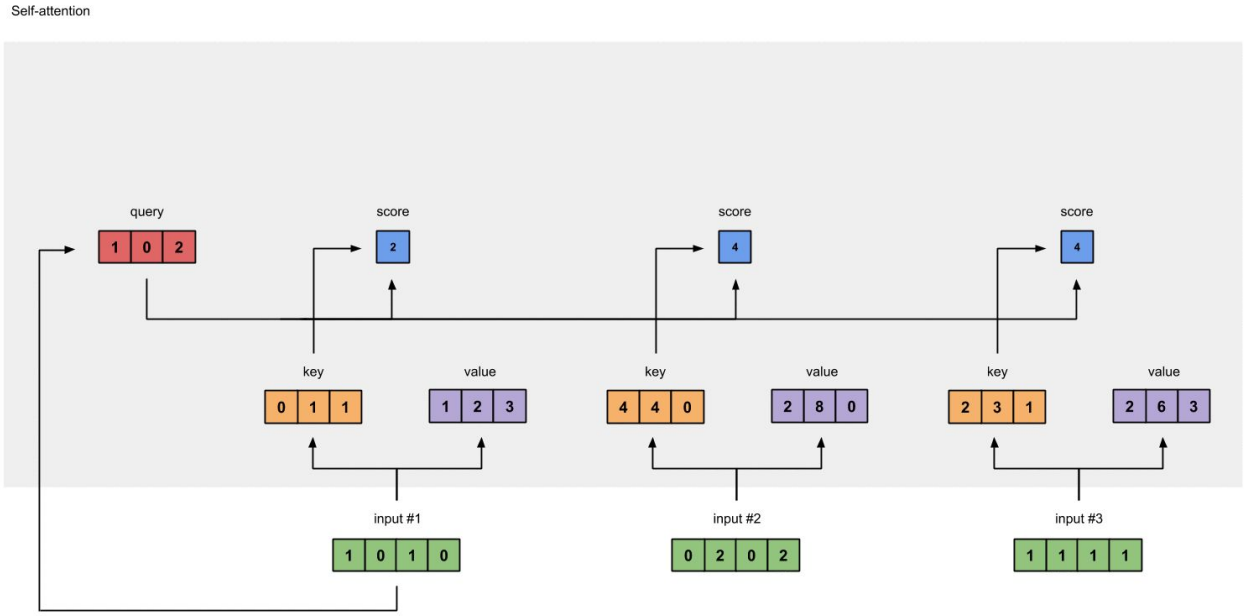
https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention steps



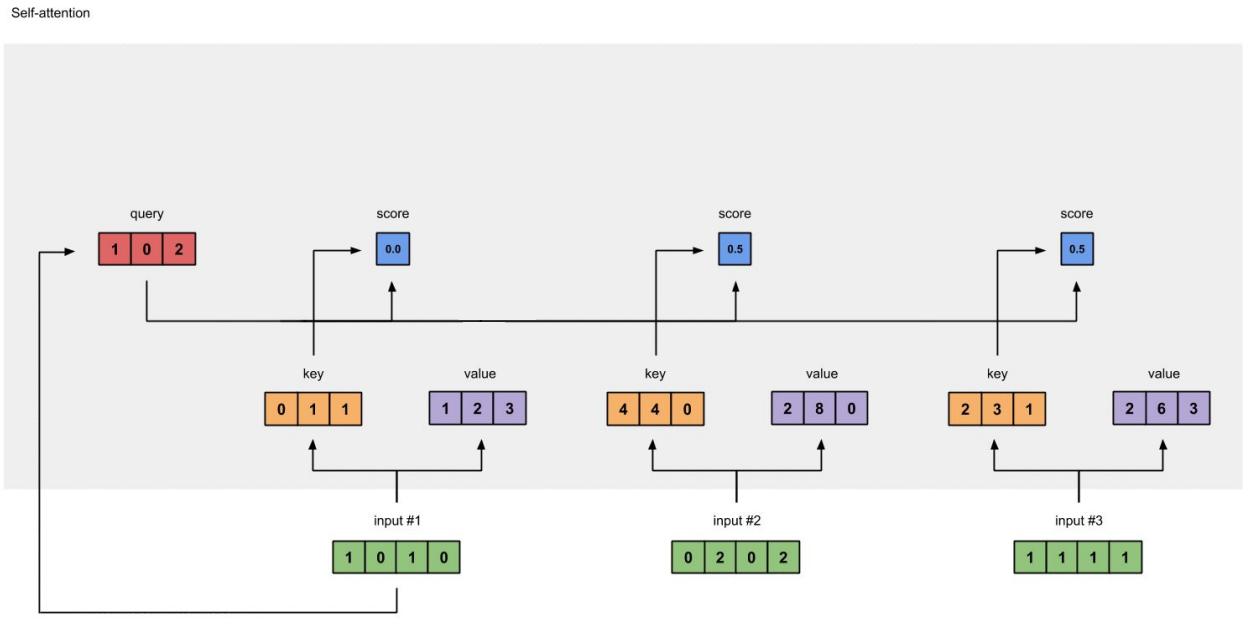
https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention steps



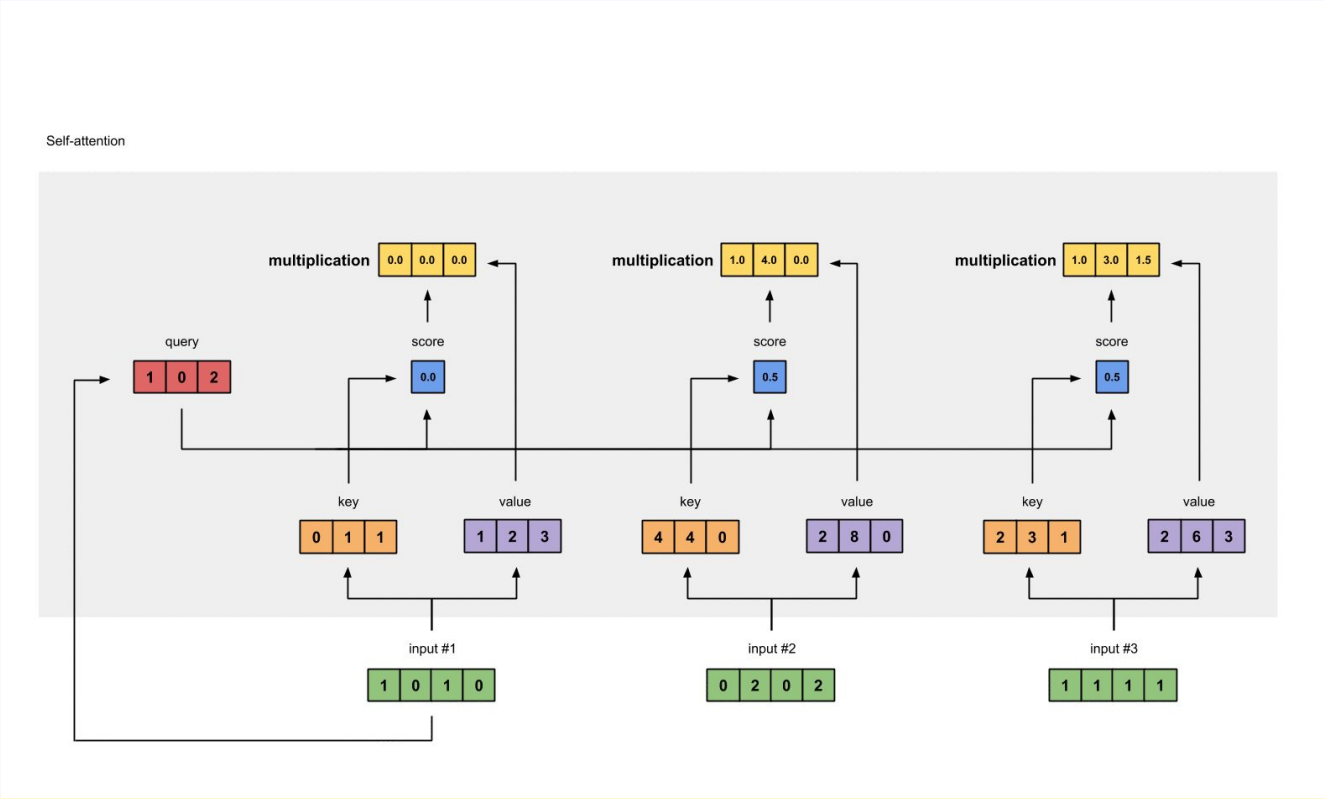
https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention steps



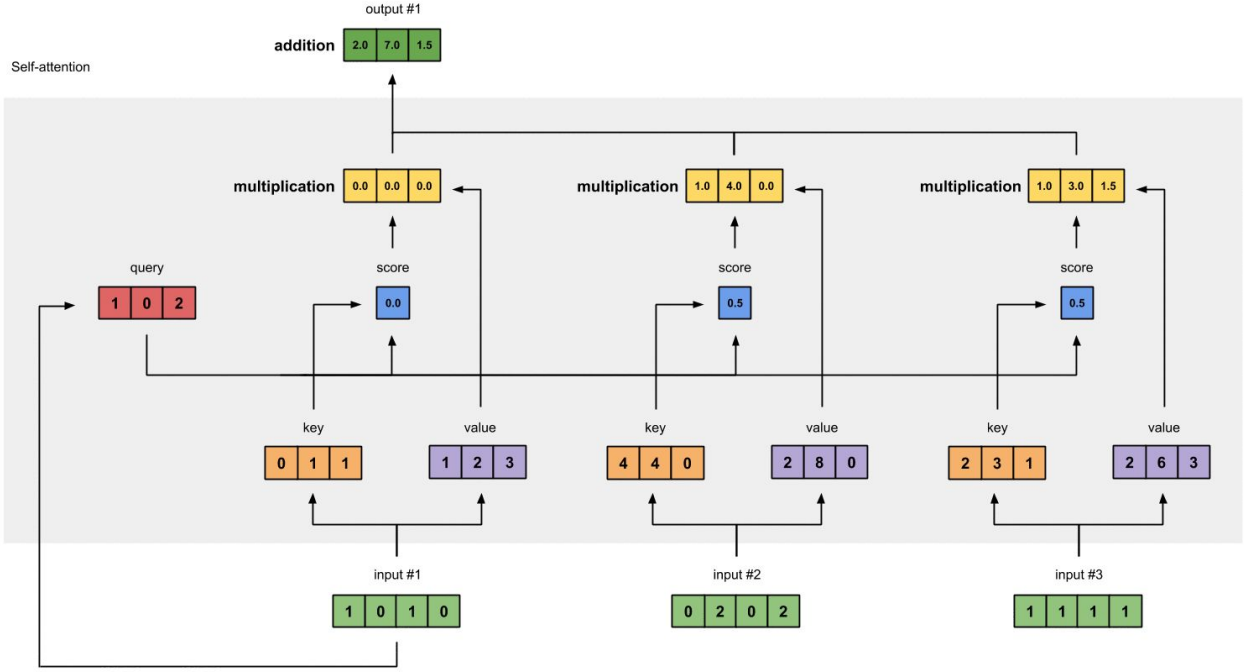
https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention steps



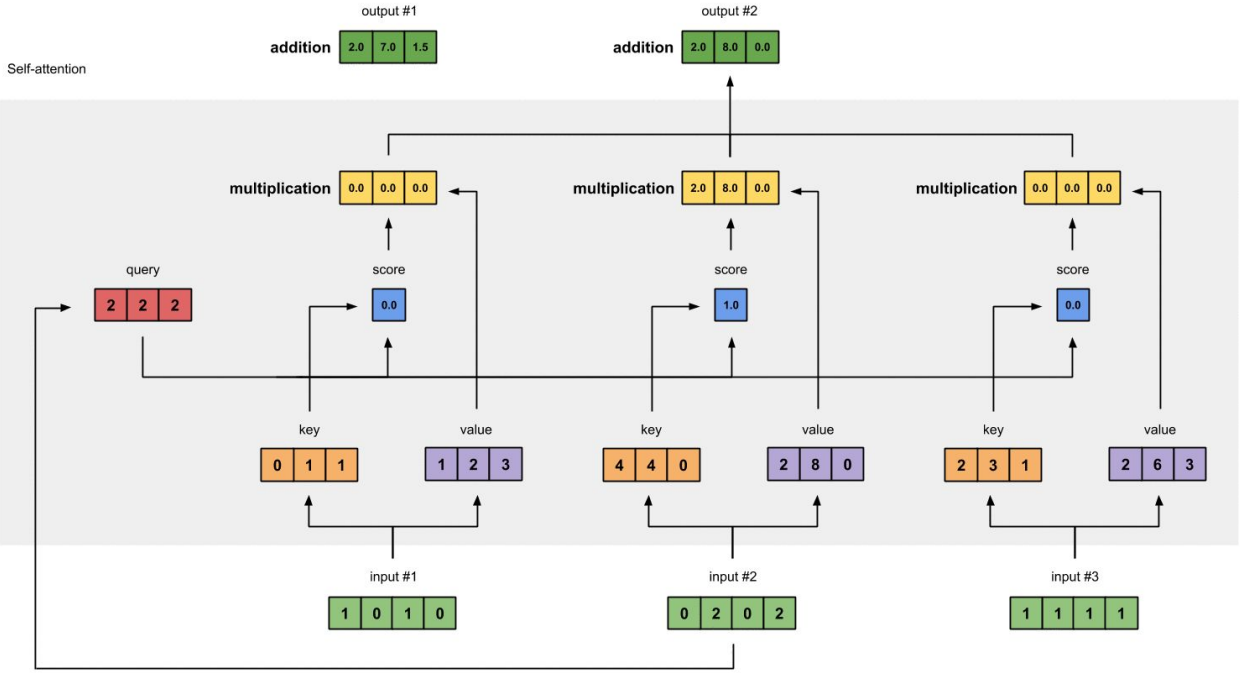
https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention steps



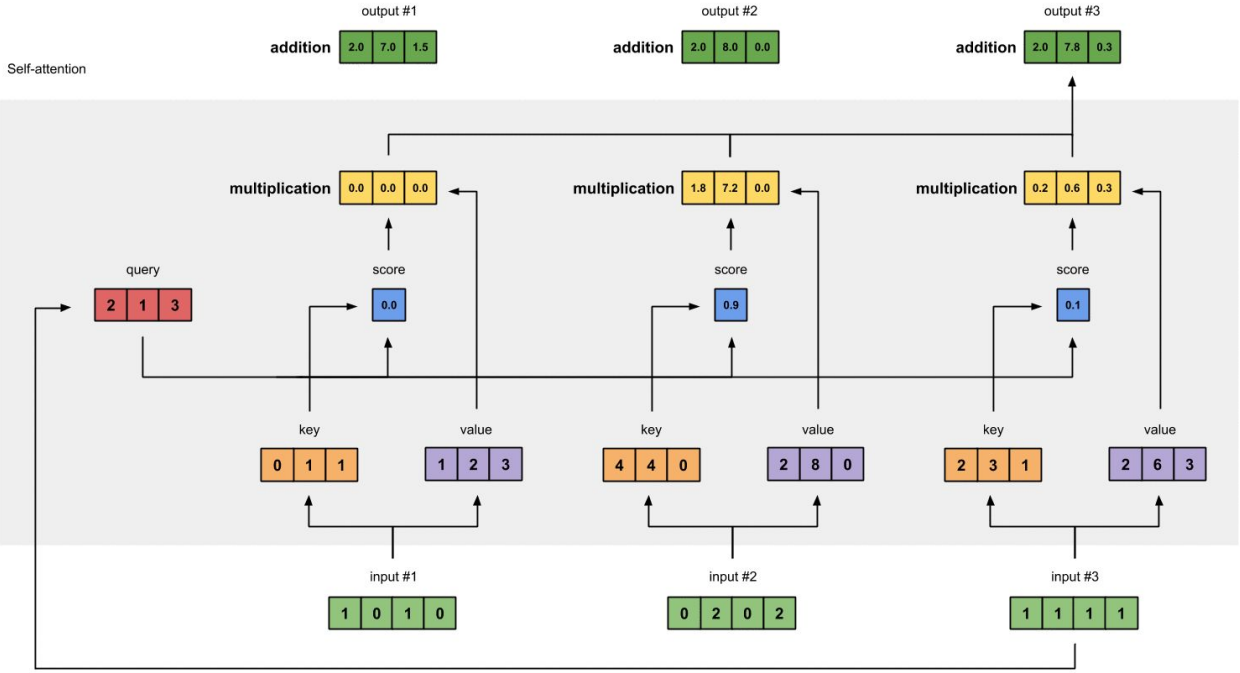
https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention steps



https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

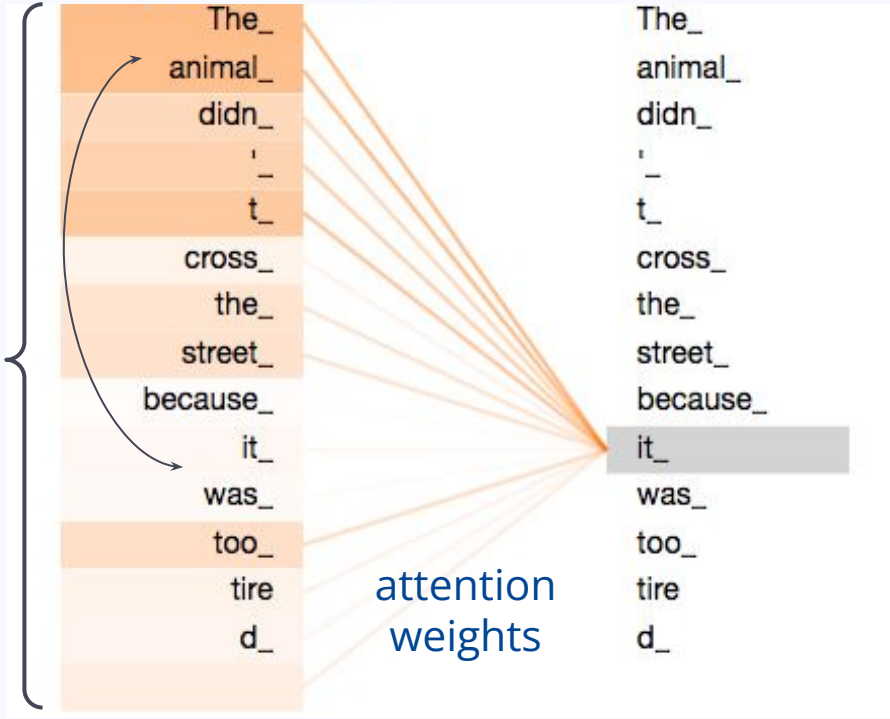
Self-attention steps



https://colab.research.google.com/drive/1rPk3ohrmVclqhH7uQ7qys4oznDdAhpzF?usp=sharing

Self-attention representations

Link to run your own visualizations!



Attention of **token "it"**:

- representations for all tokens (+ itself)
- attends to "the", "animal" the most
- also captures negation (important!)

https://jalammar.github.io/illustrated-transformer/

Self-attention: Pros & Cons

+ Great at **long-distance relations!**
+ Good at **coreference resolution**!
+ Better **contextual understanding & representations**
+ **Parallel computations** for efficiency and scalability

- Very **computationally complex**!
- Needs multiple **layers** and **heads** to be really good

2. Transformers

The path to Transformers

seq2seq models

learning input serially

The path to Transformers

seq2seq models → **seq2seq + attention**

learning input serially

learning input serially & learning important "shortcuts" of context

The path to Transformers

seq2seq models → **seq2seq + attention** → **Transformers** + **self-attention**

learning input serially

learning input serially & learning important "shortcuts" of context

parallelized learning & use of self-attention for word representations

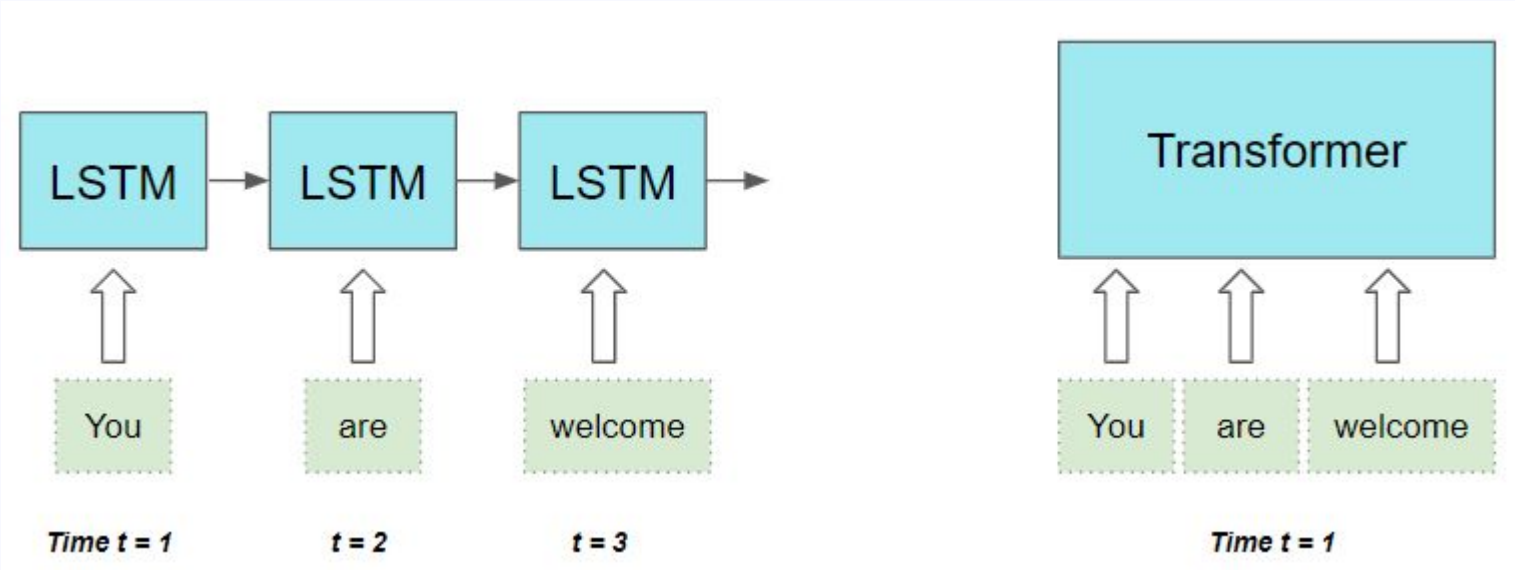
collects attention from the entire input, creates **representations** (+ **multi-headed**, i.e. many subspaces!)

ANITI

The path to Transformers

Traditional NNs: Multiple timesteps

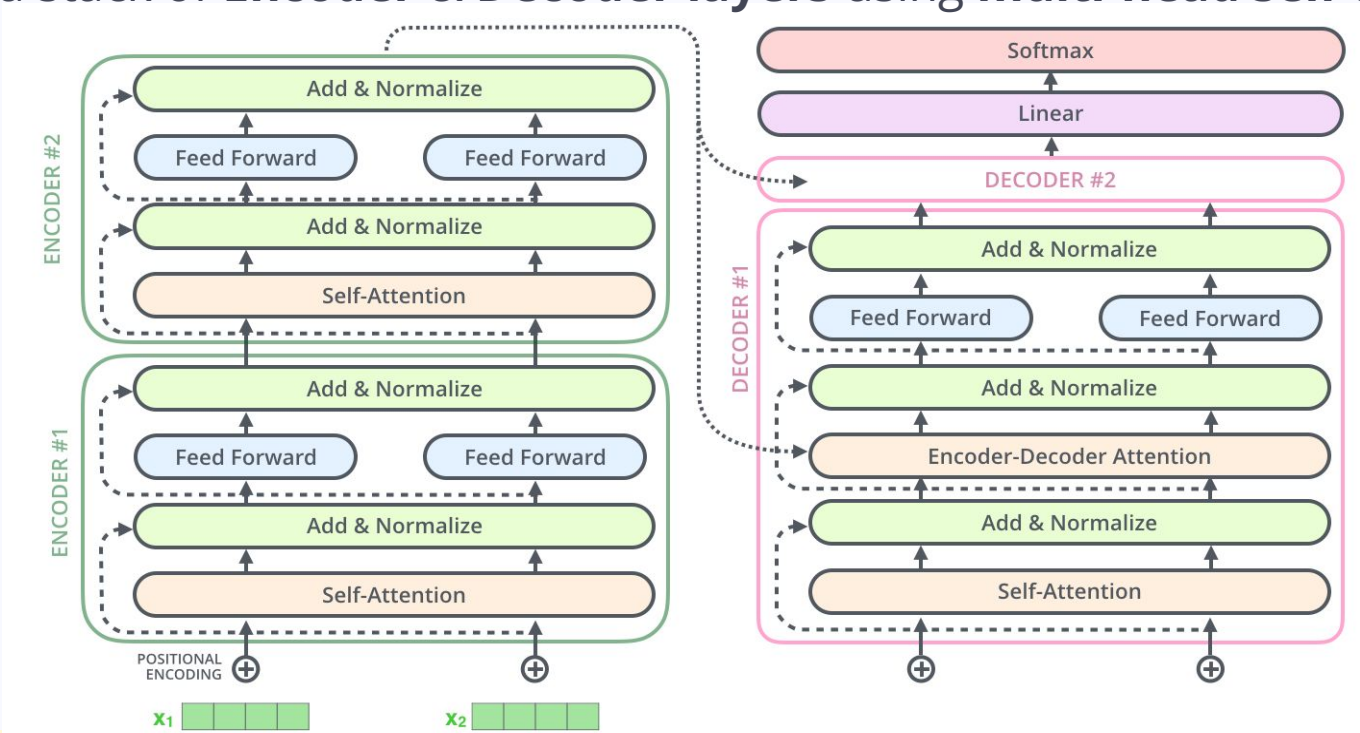
Transformers: Parallel!



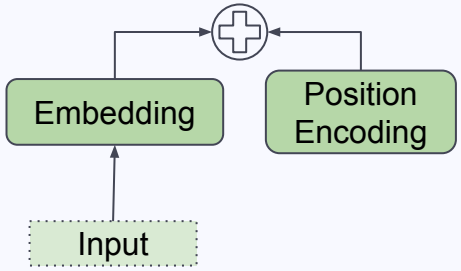
https://towardsdatascience.com/transformers-explained-visually-part-1-overview-of-functionality-95a6dd460452/

What is a Transformer?

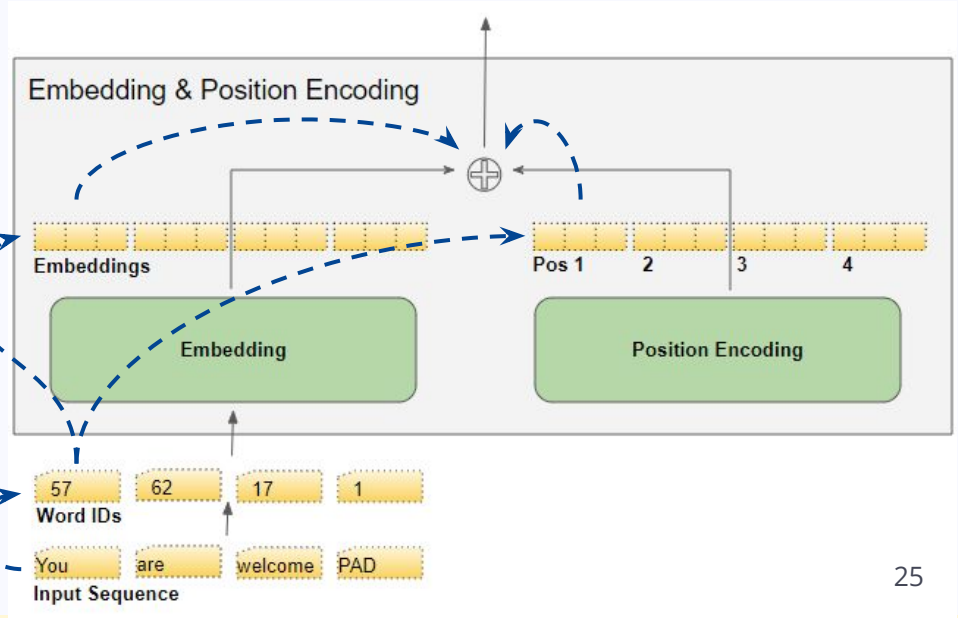
NN with a stack of **Encoder** & **Decoder layers** using **multi-head self-attention**



The Transformer's Input



- **Embedding** = meaning of the word
- **Position encoding** = position of the word
- Transformer **combines** them!

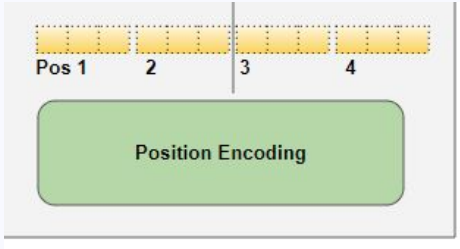


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/

Positional Encodings

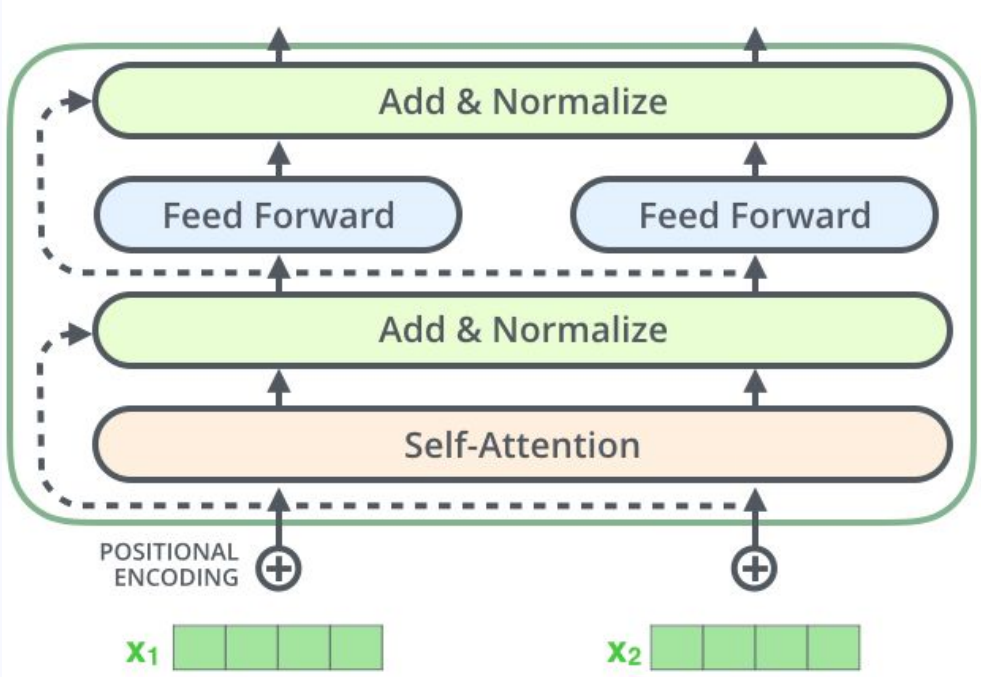
- **Traditional NNs**: sequential processing → position is evident!
- But with parallel processing in **Transformers**?

- **Positional encoding**: an independent layer from the input sequence
- Encodes the position, length and index value of token
- More complex than just `[0, 1, 2, 3, ...]` …



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/

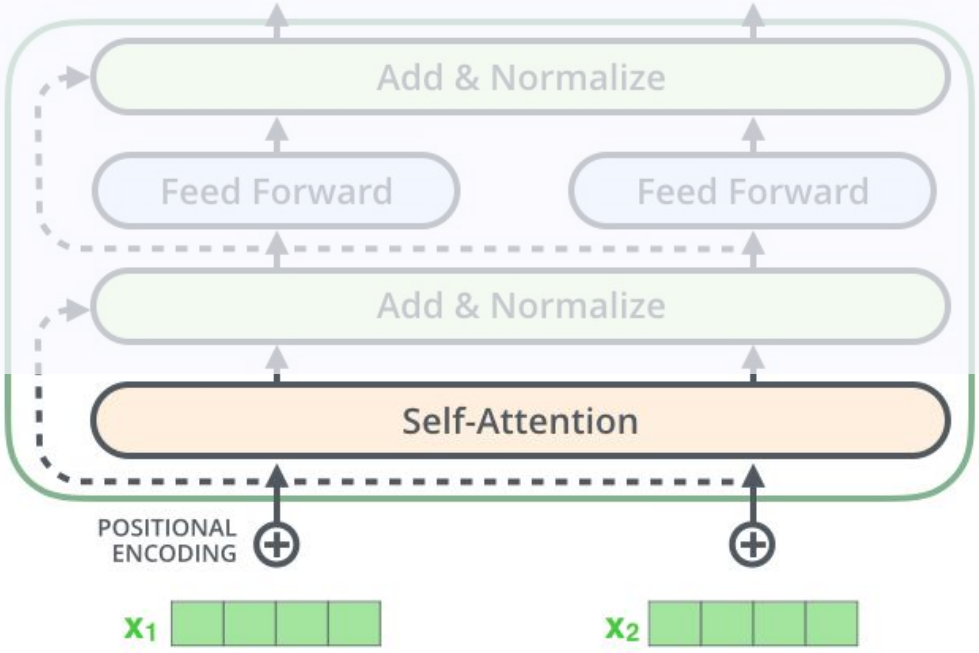
The Transformer's Encoder #1



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Encoder #1

1. **Multi-headed Self-attention**

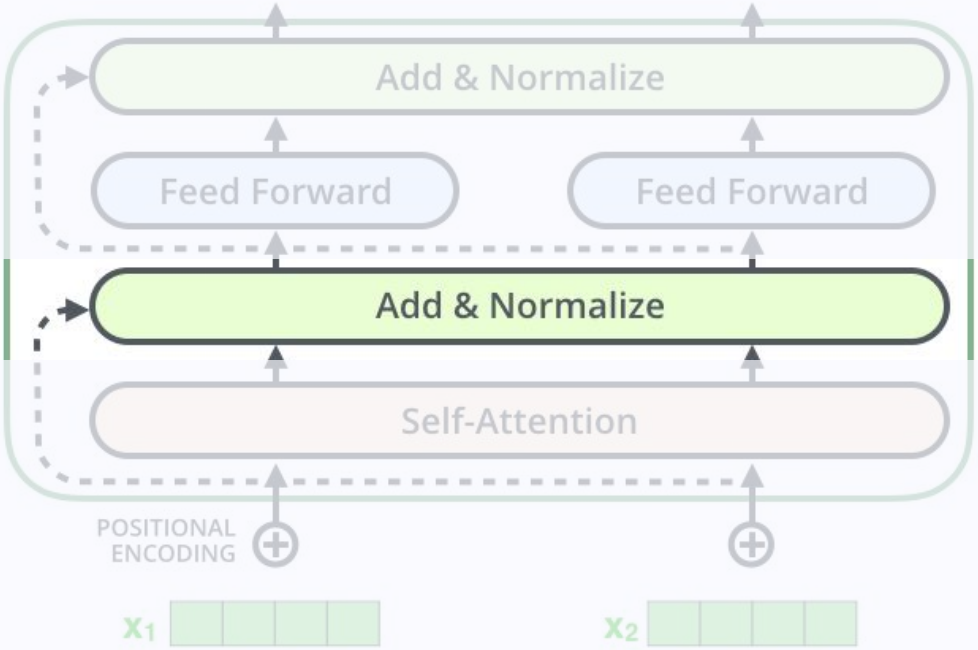


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Encoder #1

1. **Multi-headed Self-attention**
2. **Layer Normalization**:

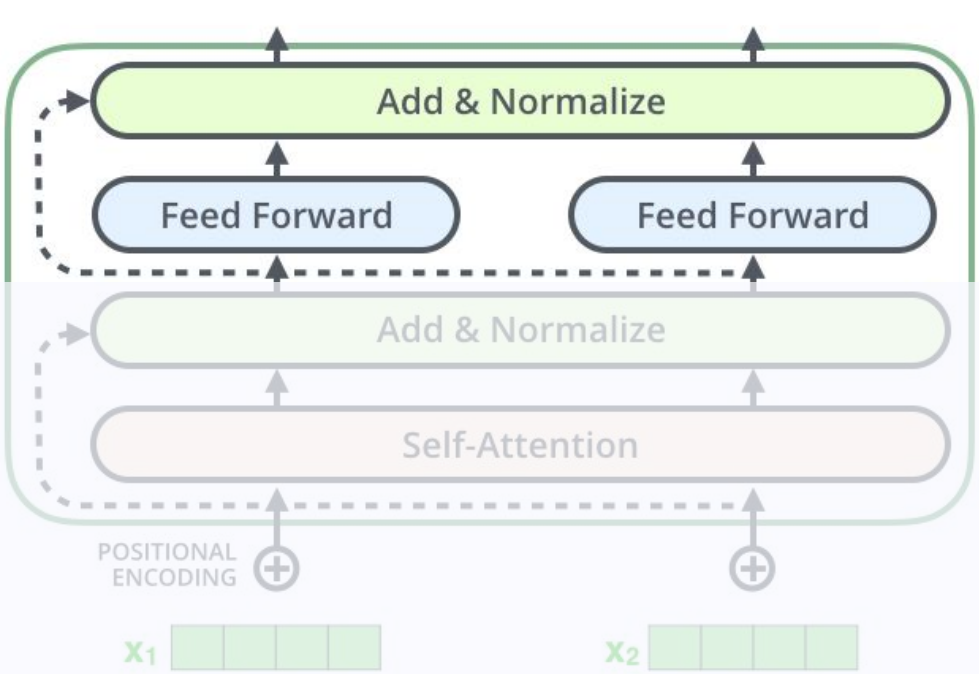
   stabilize the hidden states



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Encoder #1

1. **Multi-headed Self-attention**
2. **Layer Normalization**:
   stabilize the hidden states
3. **Feedforward** neural network
4. More **layer normalization**!

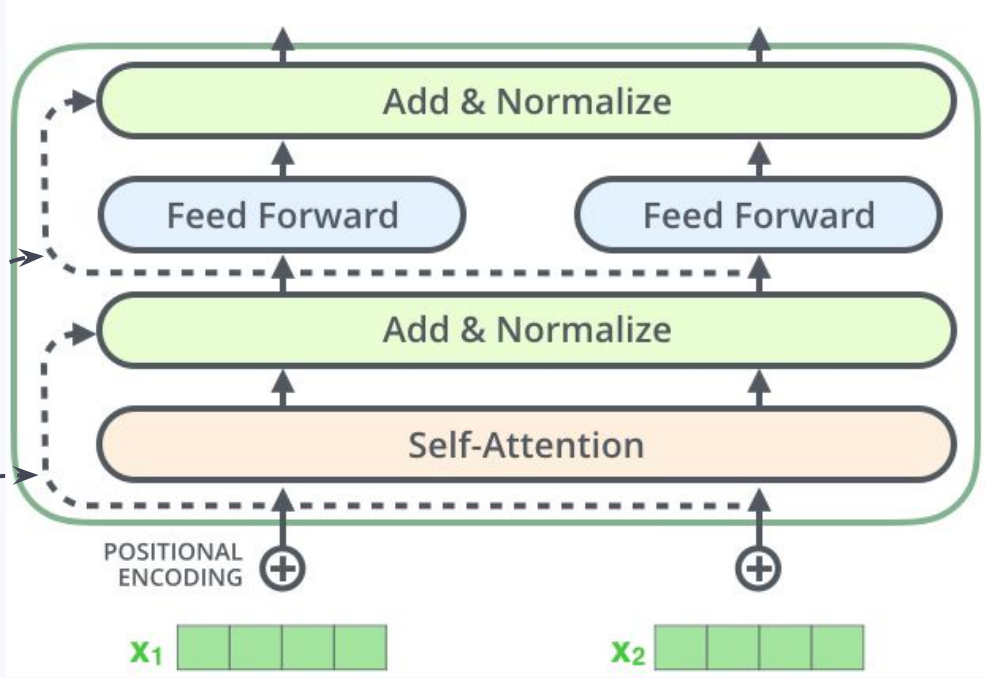


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Encoder #1

1. **Multi-headed Self-attention**
2. **Layer Normalization**:
   stabilize the hidden states
3. **Feedforward** neural network
4. More **layer normalization**!

Residual connection:
pass gradients through net →
no vanishing gradients!

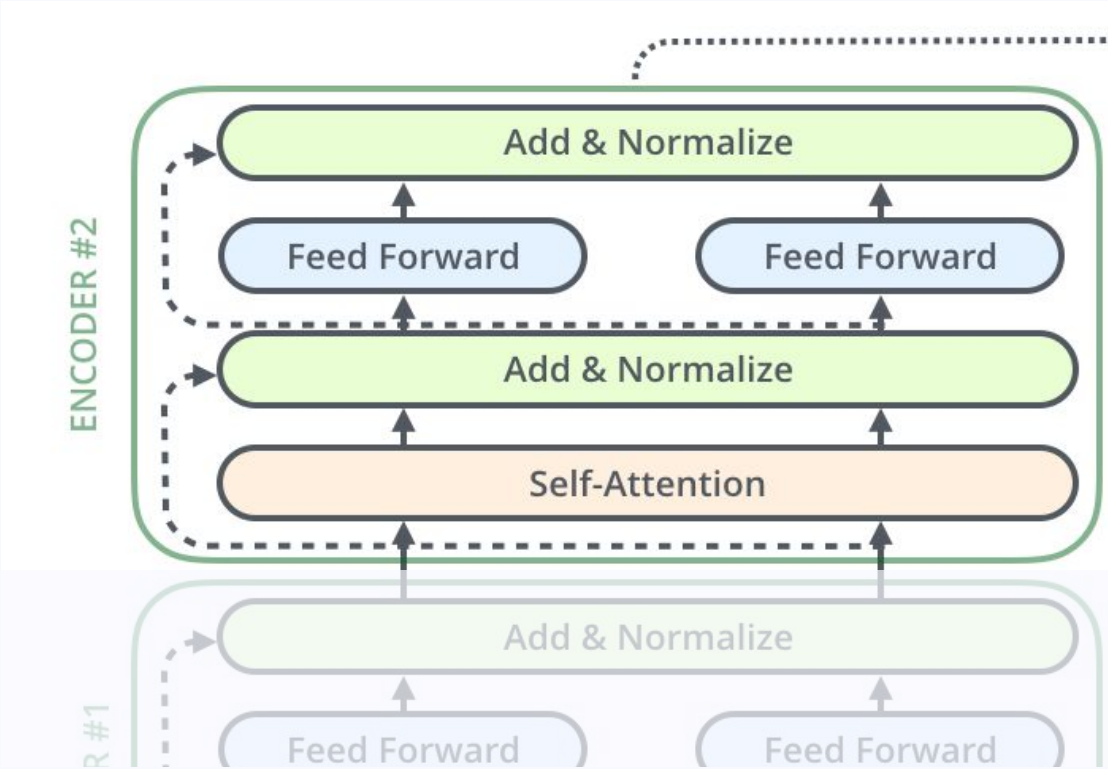


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Encoder #2

Identical to Encoder #1!

(but receives Encoder #1's output, instead of the OG input sequence)

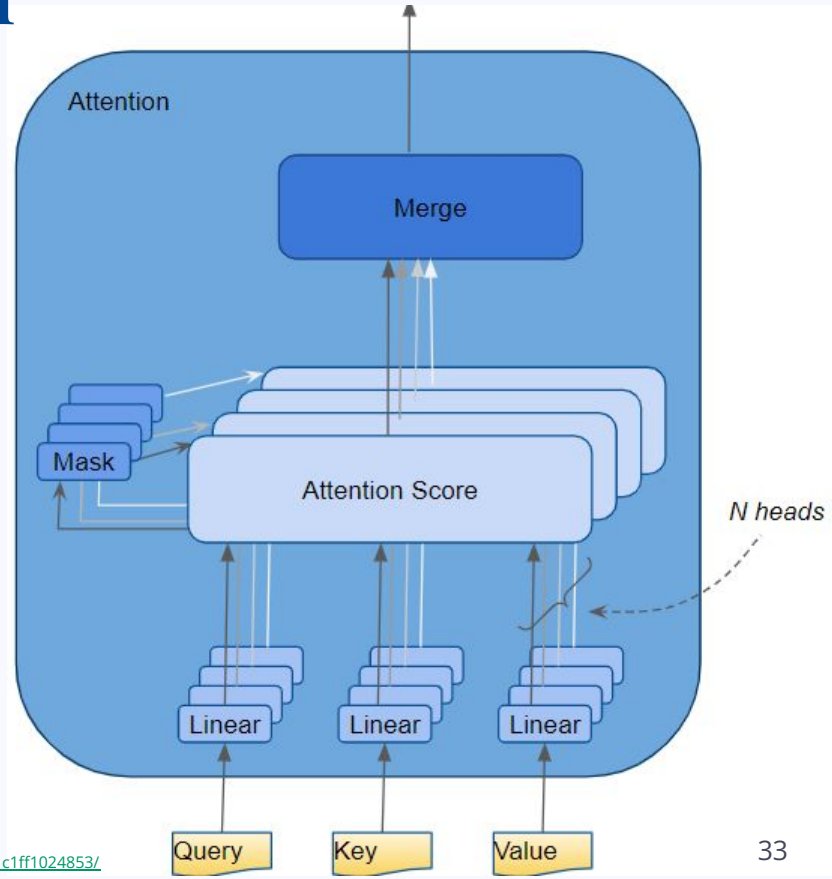


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

Multi-head Self-attention

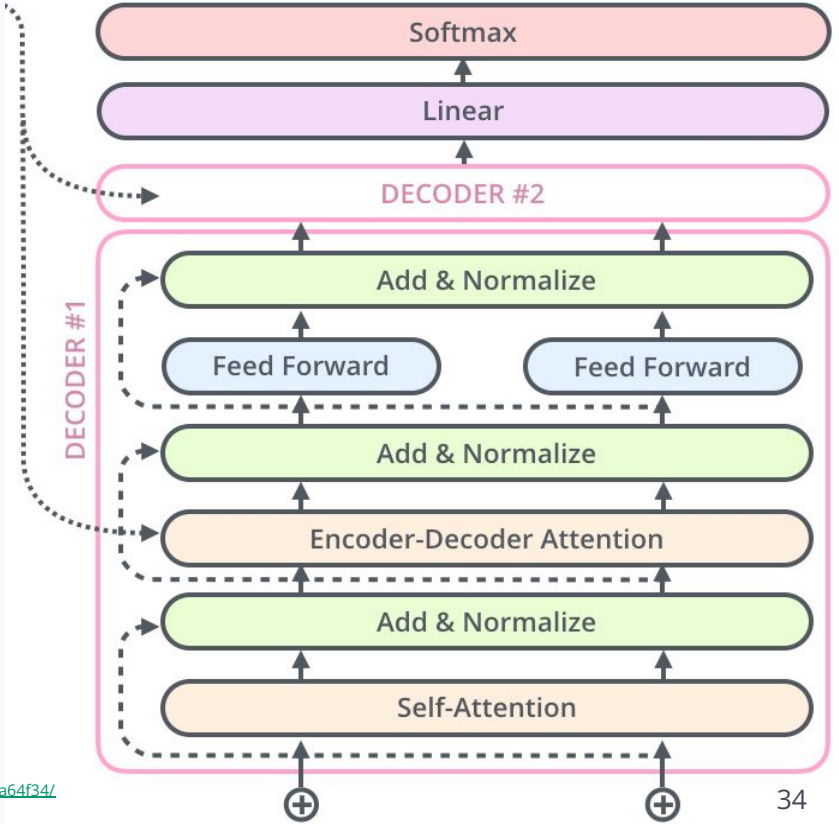
Inside the encoder, not between enc-dec!

1. Start with keys:values - queries
2. **Split** them in N parts (N = # of heads)
3. **Each head** computes
4. **Combine** heads' attention scores → **final attention score**



https://towardsdatascience.com/transformers-explained-visually-part-3-multi-head-attention-deep-dive-1c1ff1024853/

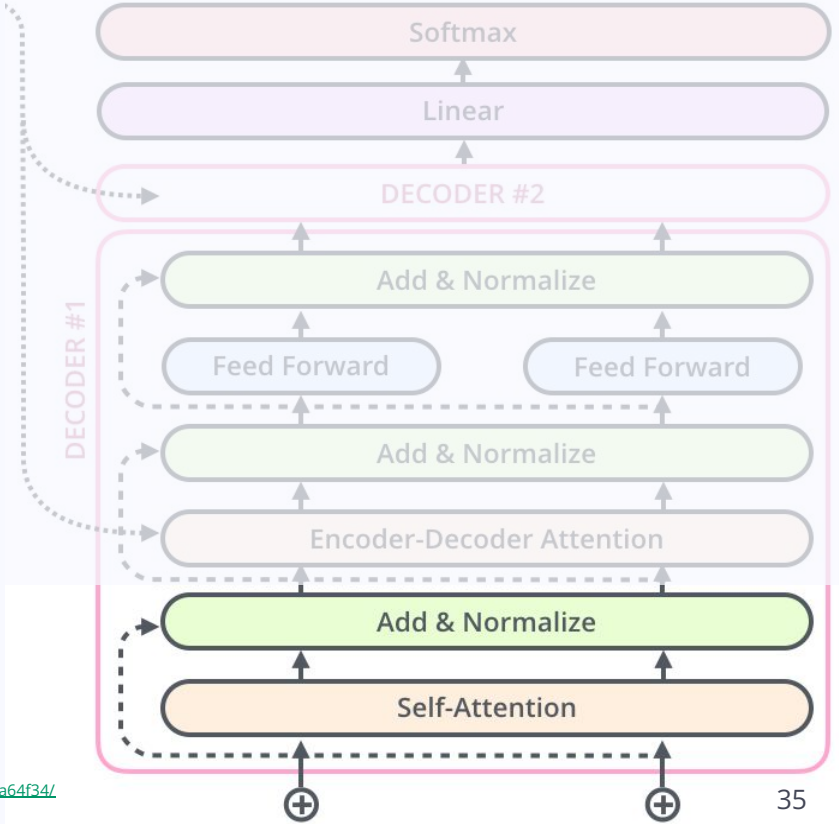
The Transformer's Decoder #1



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Decoder #1

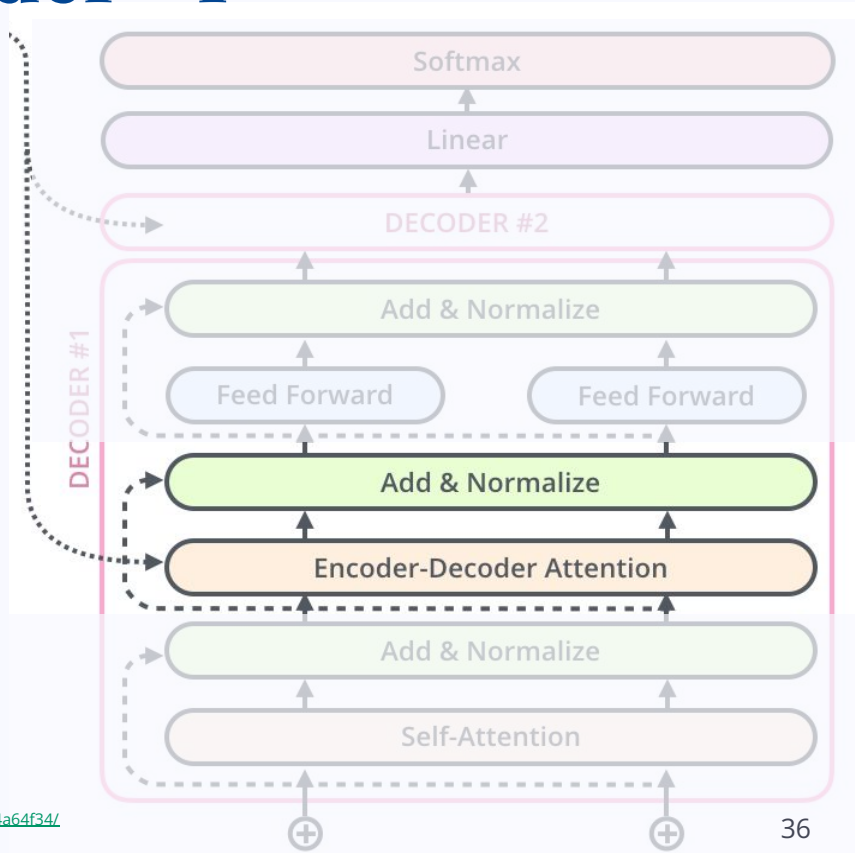
1. Multi-head self-attention: **only** attend to earlier positions in the sequence
2. Layer normalization



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Decoder #1

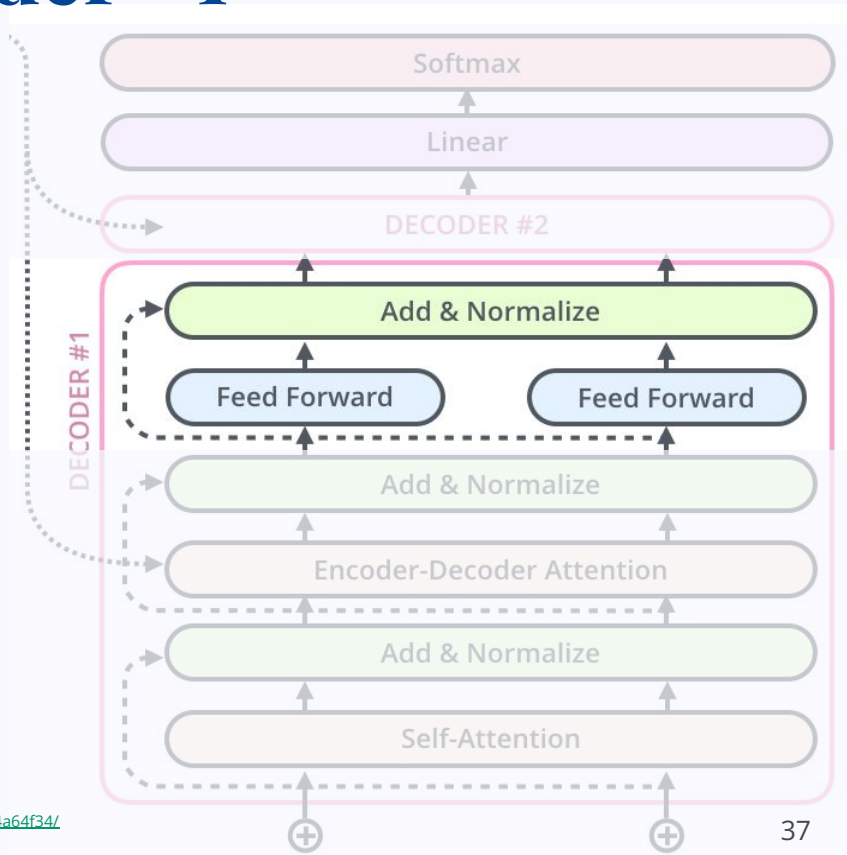
1. Multi-head self-attention: **only** attend to earlier positions in the sequence
2. Layer normalization
3. **Encoder-decoder attention layer**: combine encoder output (**.....**) + output of previous self-attention layer



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Decoder #1

1. Multi-head self-attention: **only** attend to earlier positions in the sequence
2. Layer normalization
3. **Encoder-decoder attention layer**: combine encoder output + output of previous self-attention layer
4. Feed-forward layers

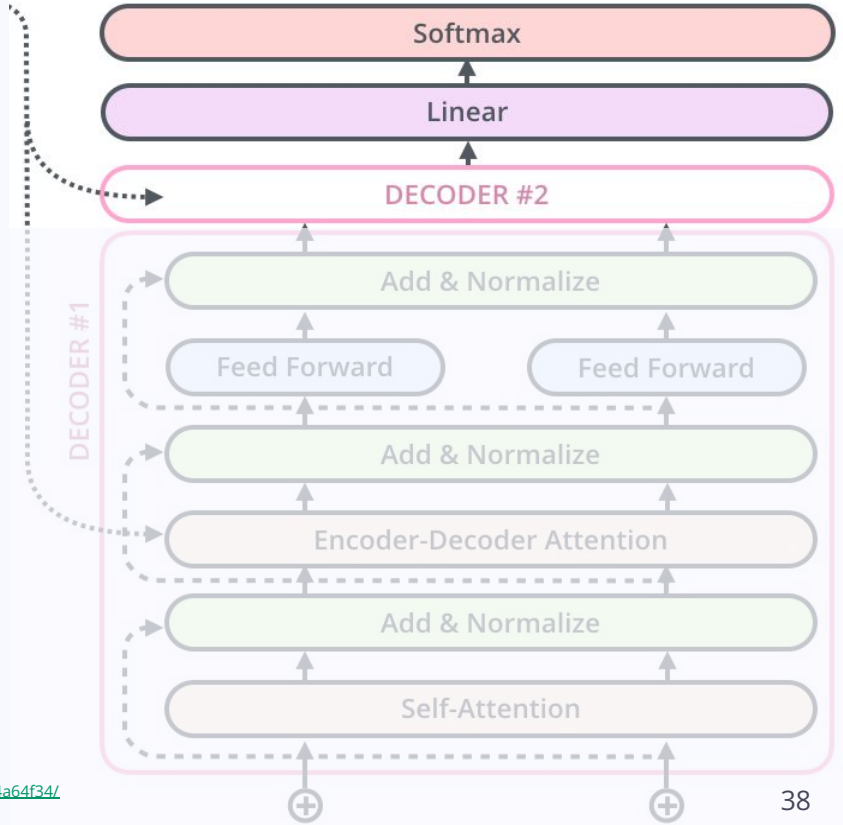


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

The Transformer's Decoder #2 etc.

- **Decoder #2**: Identical to Decoder #1
- **Linear layer**: assign probability to each token of the target vocabulary (`logits`)
- **Softmax layer**: turn probabilities 0-1 (`log probabilities`)

- **Output** = highest probability token!

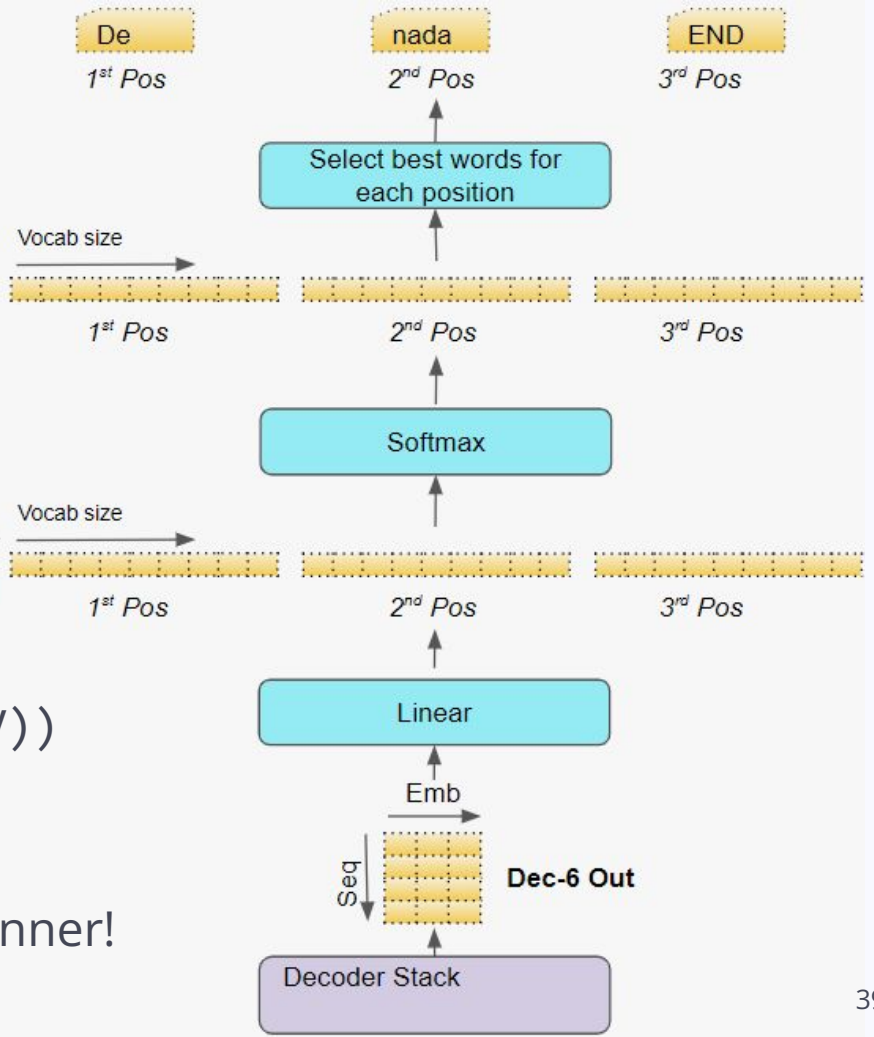


https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/
https://jalammar.github.io/illustrated-transformer/

Generate Output

1. Decoder passes to Linear
2. Linear: for every word in V,
   score of word in <u>each</u> position
   (`logits: (len(output), len(V))`)
3. Softmax: score → probability (0-1)
   (`log probs: (len(output), len(V))`)

Word with highest prob. in position x: Winner!



https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/

Training & Loss Function

We still use backpropagation to calculate loss!

https://towardsdatascience.com/transformers-explained-visually-part-2-how-it-works-step-by-step-b49fa4a64f34/

3. (Large) Language Models

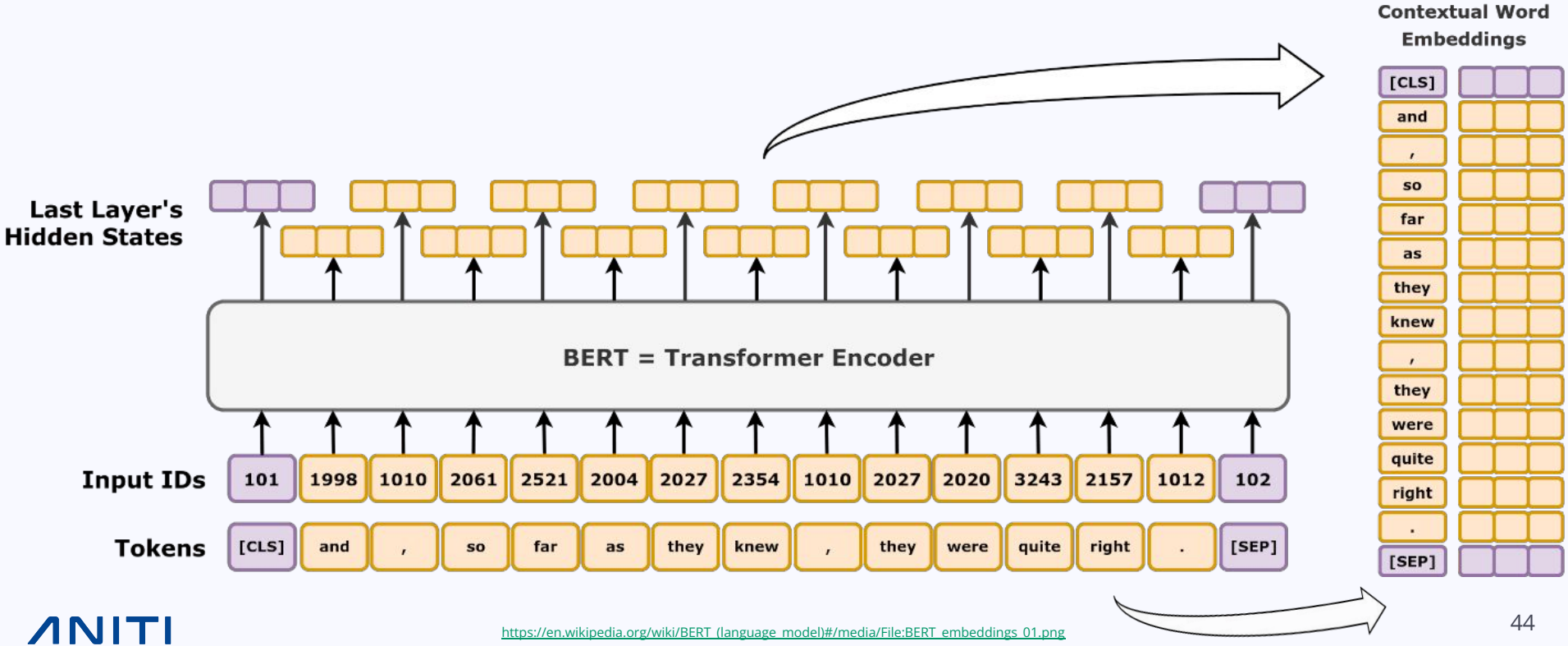
BERT

LLM spotlight: BERT

- **B**idirectional : bidirectional self-attention!
- **E**ncoder : its output = our context vectors!
- **R**epresentations : token embeddings relative to their context!
- from **T**ransformers : uses a transformer NN!

Training goals: Masked Language Modeling & Next Sentence Prediction

Masked Language Modeling



https://en.wikipedia.org/wiki/BERT_(language_model)#/media/File:BERT_embeddings_01.png

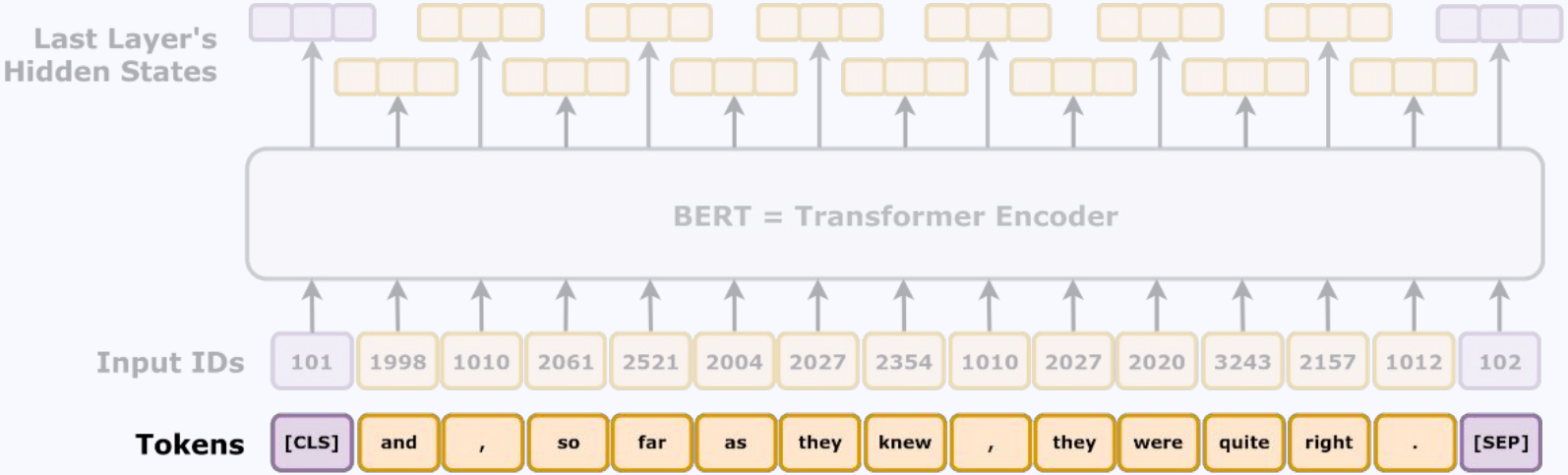
Masked Language Modeling

BERT's special tokens:

[CLS] = "Classify"

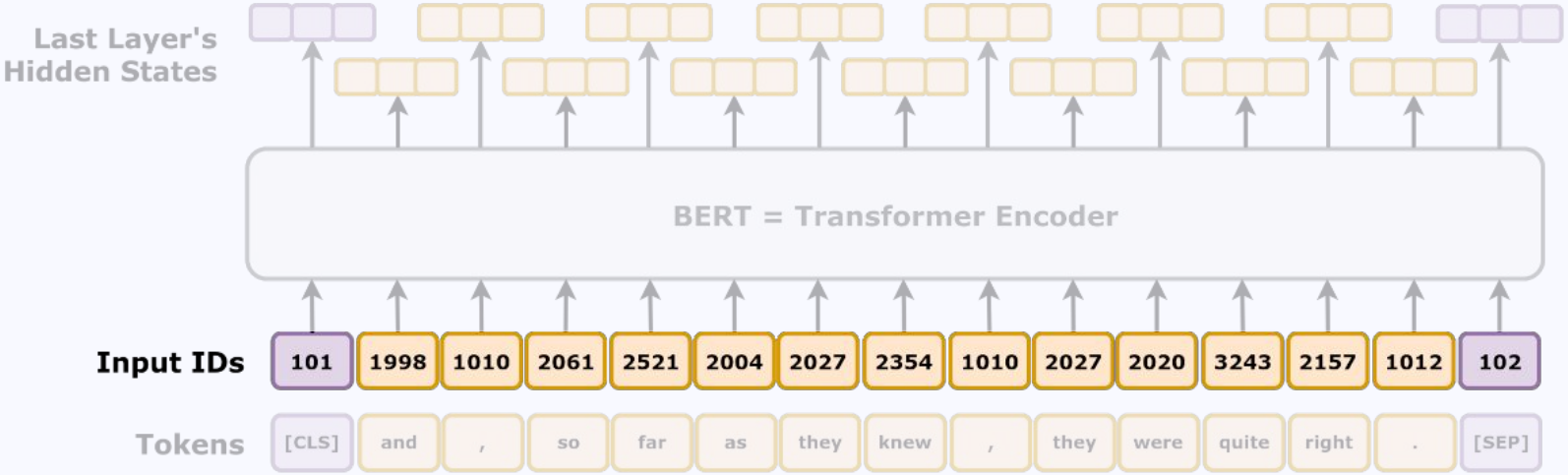
[SEP] = Separates phrases

[MASK] = Hides a token



https://en.wikipedia.org/wiki/BERT_(language_model)#/media/File:BERT_embeddings_01.png

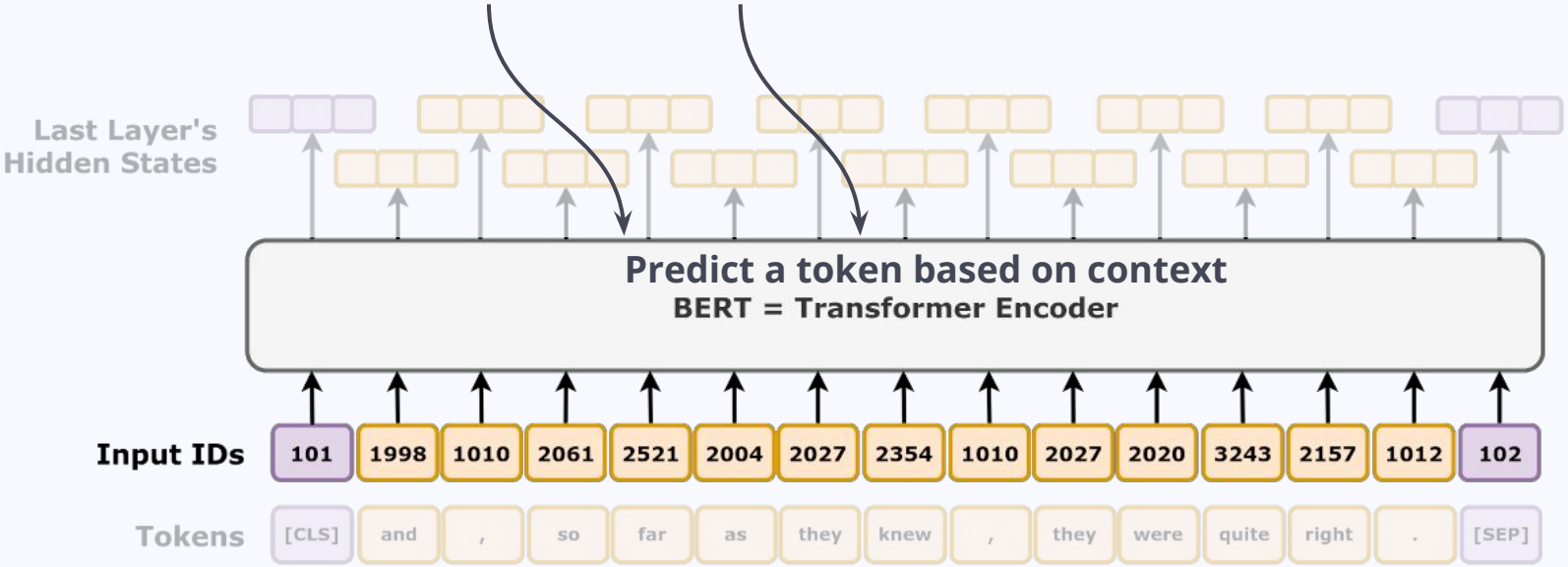
Masked Language Modeling



https://en.wikipedia.org/wiki/BERT_(language_model)#/media/File:BERT_embeddings_01.png

Masked Language Modeling

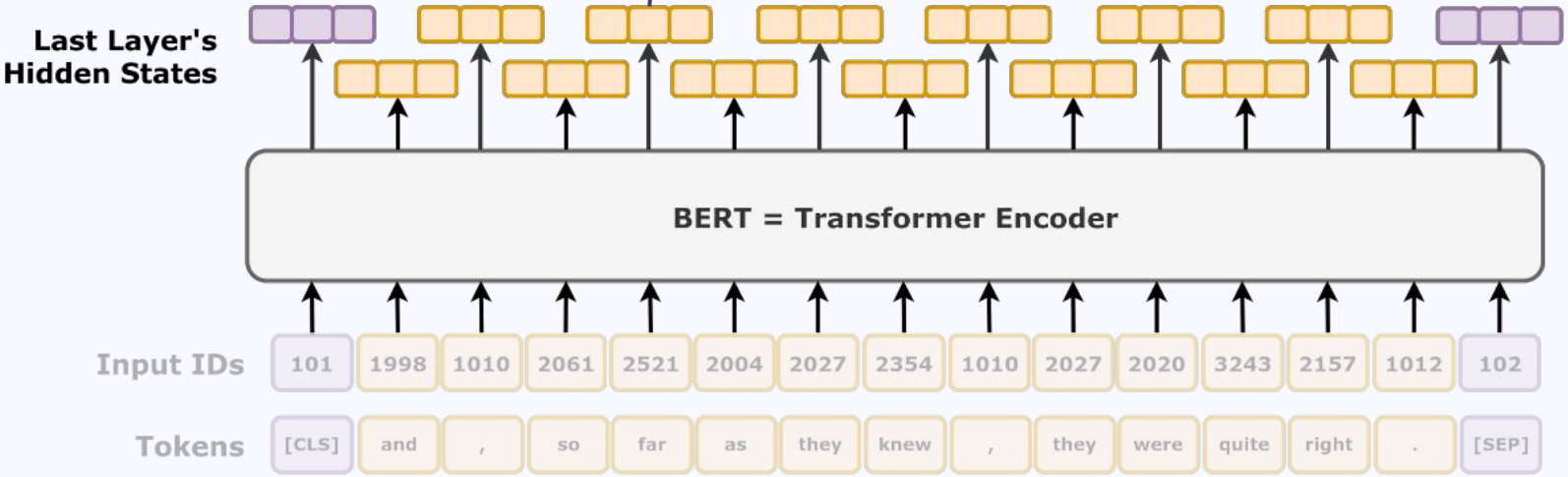
bert-base: **12** layers x **12** attention heads = 144 representations



https://en.wikipedia.org/wiki/BERT_(language_model)#/media/File:BERT_embeddings_01.png

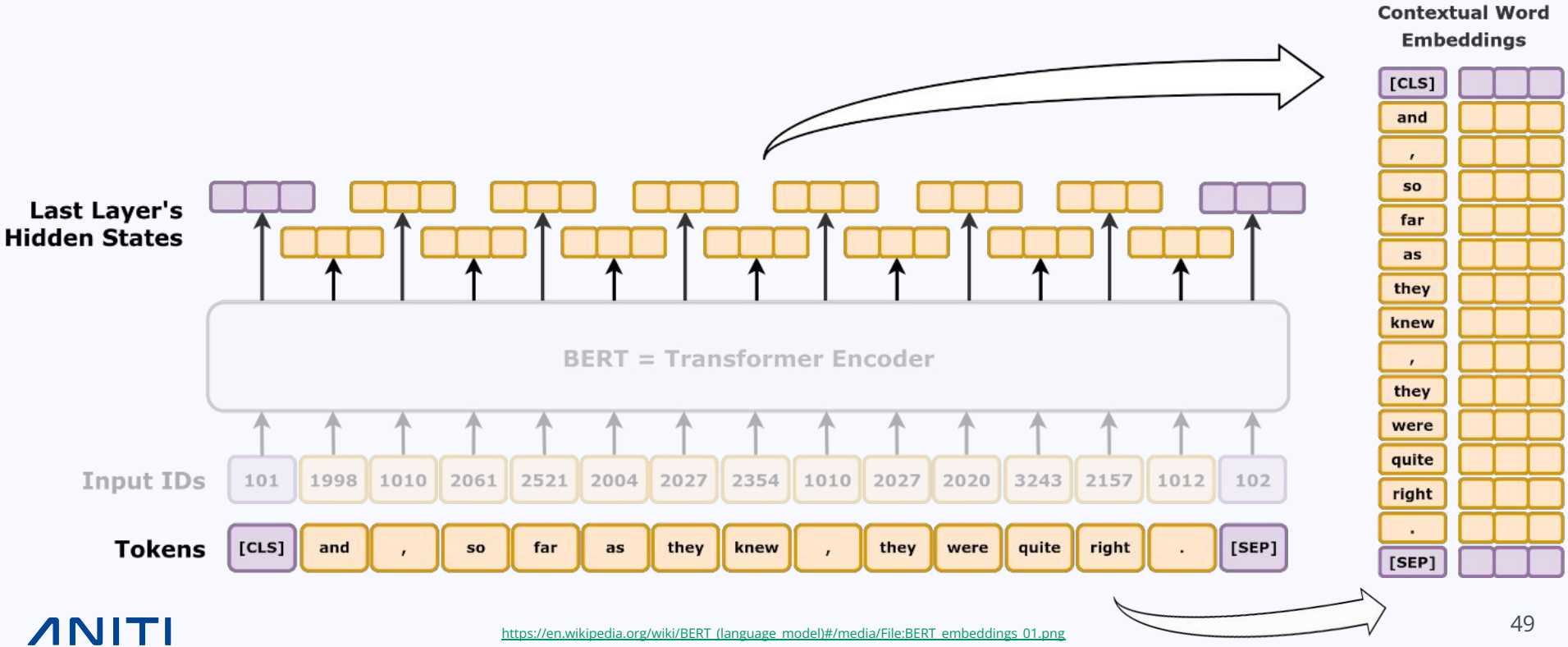
Masked Language Modeling

probability of "far" in the specific sentence
"and, so ___ as they knew, they were quite right."
specifically in the 5th position



https://en.wikipedia.org/wiki/BERT_(language_model)#/media/File:BERT_embeddings_01.png

Masked Language Modeling



https://en.wikipedia.org/wiki/BERT_(language_model)#/media/File:BERT_embeddings_01.png

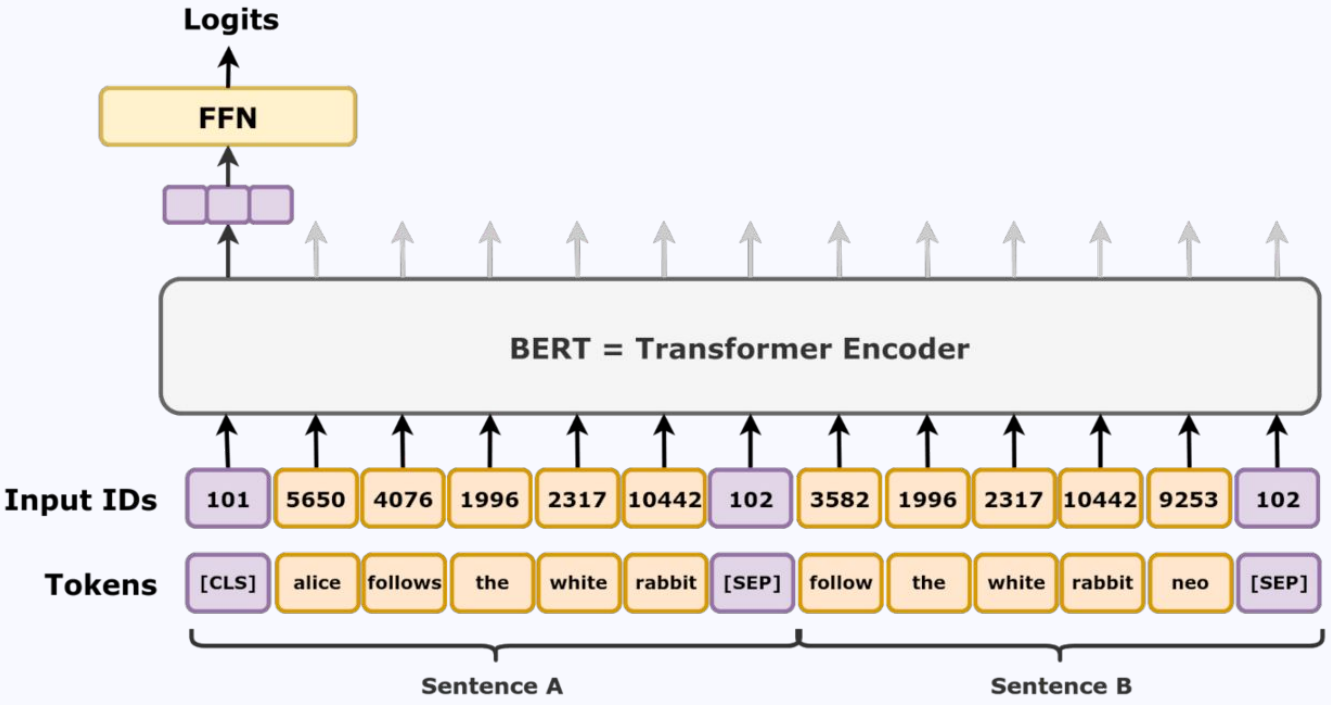
Next Sentence Prediction

BERT's special tokens:

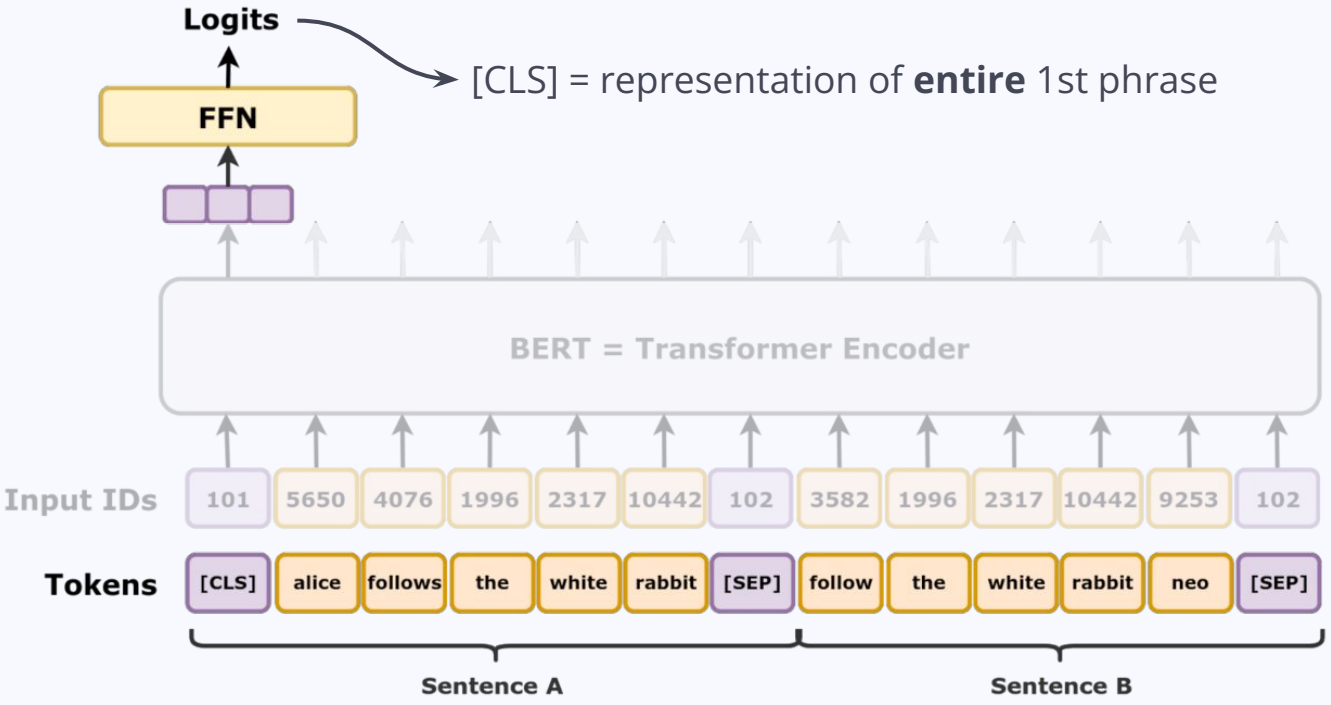
[CLS] = "Classify"

[SEP] = Separates phrases

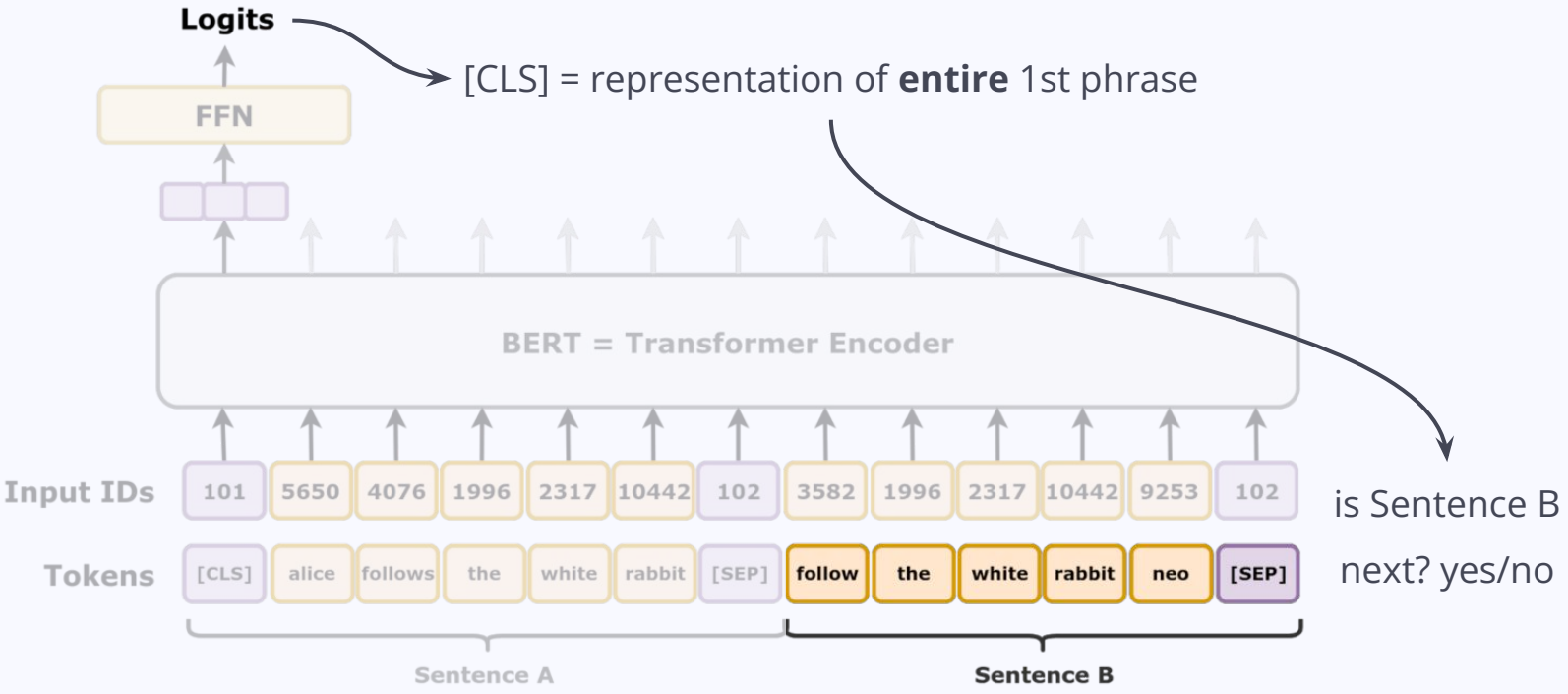
[MASK] = Hides a token



Next Sentence Prediction



Next Sentence Prediction



BERT representations

- [MASK] words to learn about them from context only!
- Cloze tasks: fill in the blanks!

I love eating _____ salad.

BERT representations

- [MASK] words to learn about them from context only!
- Cloze tasks: fill in the blanks!

I love eating ____ salad.

- spinach, Caesar, etc.
- fresh
- my mom's

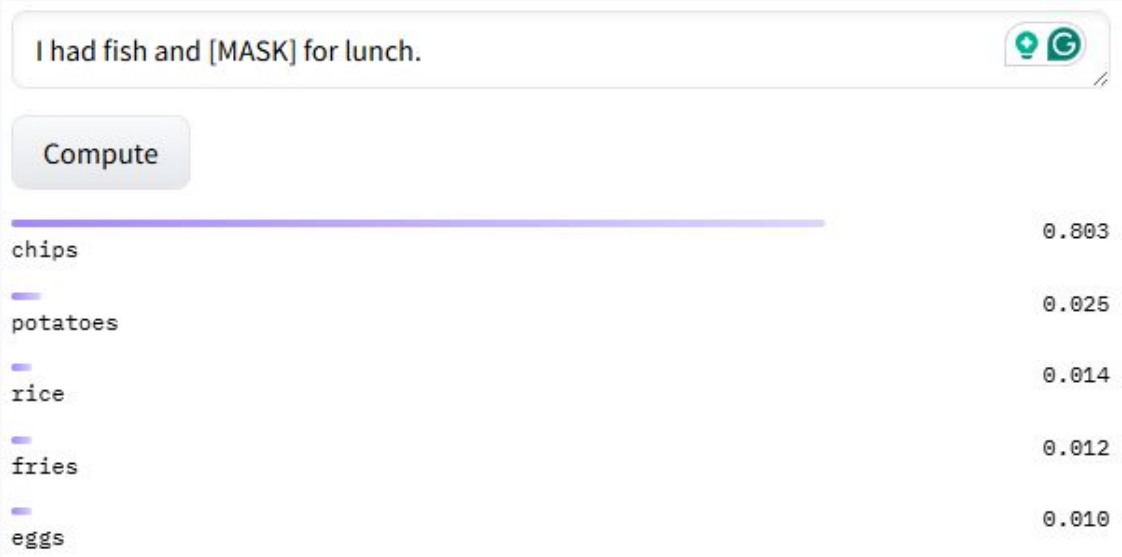
https://huggingface.co/google-bert/bert-base-uncased

BERT representations

- [MASK] words to learn about them from context only!
- Cloze tasks: fill in the blanks!

I had fish and ___ for lunch.

“fish and chips” is a MWE!



https://huggingface.co/google-bert/bert-base-uncased

What do BERT's embeddings know?

- Do they look like **traditional embeddings** (distribution, transformations)?
  - Yes… maybe in the higher layers

- Do they have **syntactic information**?
  - Bidirectionality really helped!
  - Parts of speech, syntactic chunks and roles, but not distant relations
  - No full syntactic trees, but syntactic transformations and dependencies
  - Bad with negation and with "bad" input

Rogers, A., Kovaleva, O., & Rumshisky, A. (2020). A primer in bertology: What we know about how BERT works. https://aclanthology.org/2020.tacl-1.54.pdf

What do BERT's embeddings know?

- Do they have **semantic information**?
  - Some knowledge of semantic roles, entity types, relations, proto-roles
  - Can't generalize!

- Do they have **world knowledge**?
  - Fills the blanks successfully, but not enough!
  - Bad at inference, bias?!

Rogers, A., Kovaleva, O., & Rumshisky, A. (2020). A primer in bertology: What we know about how BERT works. https://aclanthology.org/2020.tacl-1.54.pdf

BERT variations

- Bigger models: `bert-large-cased`
- Smaller models: `distilbert-base-cased`
- Improvements on training: `roberta`

and many, many more in different languages, sizes, tasks...

- BERT etc. models trained for specific NLP tasks: on Hugging Face 🤗!
  NER, Question answering, Summarization, Sentiment classification, etc.
- BERT for finance, biology, etc. also on Hugging Face!

GPT

Transformer spotlight: GPT

- **Generative**: Auto-regression uses previous output sequence as input
- **Pretrained**: Just decoding!
- **Transformer**: Using Transformer blocks for decoding

https://jalammar.github.io/illustrated-gpt2/

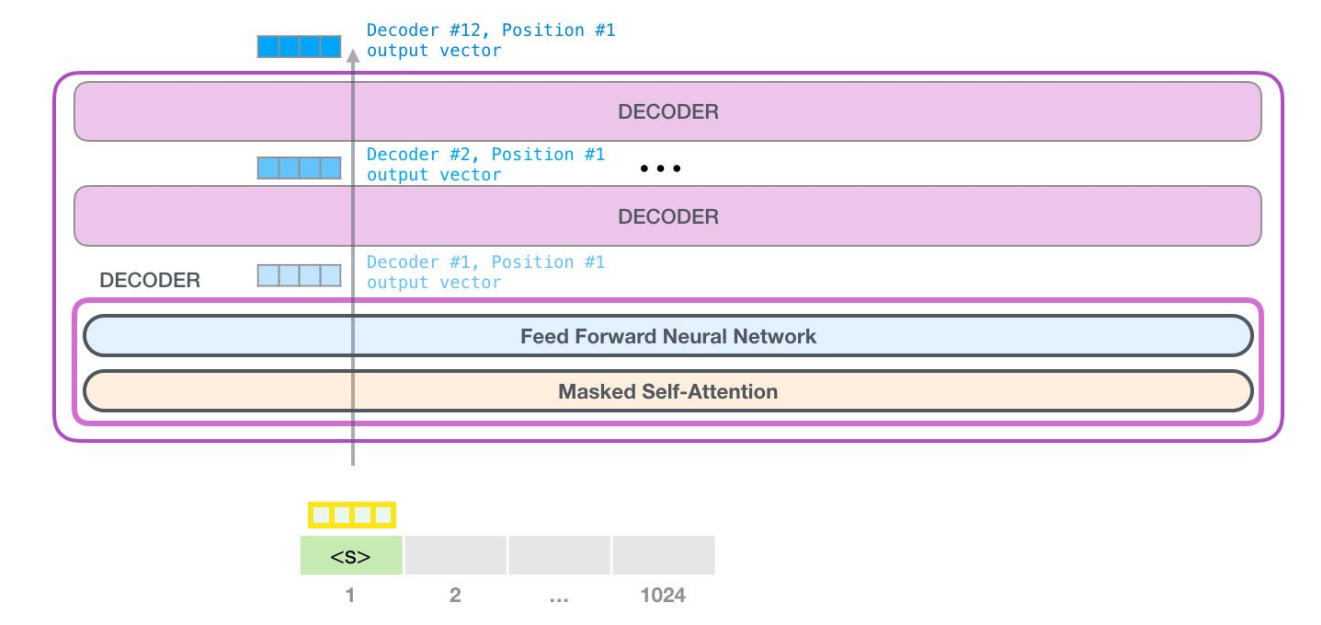
GPT Inputs

Token embeddings + **Positional encodings** = embedding of token X in position Y

https://jalammar.github.io/illustrated-gpt2/

GPT: Through the Decoder Stacks

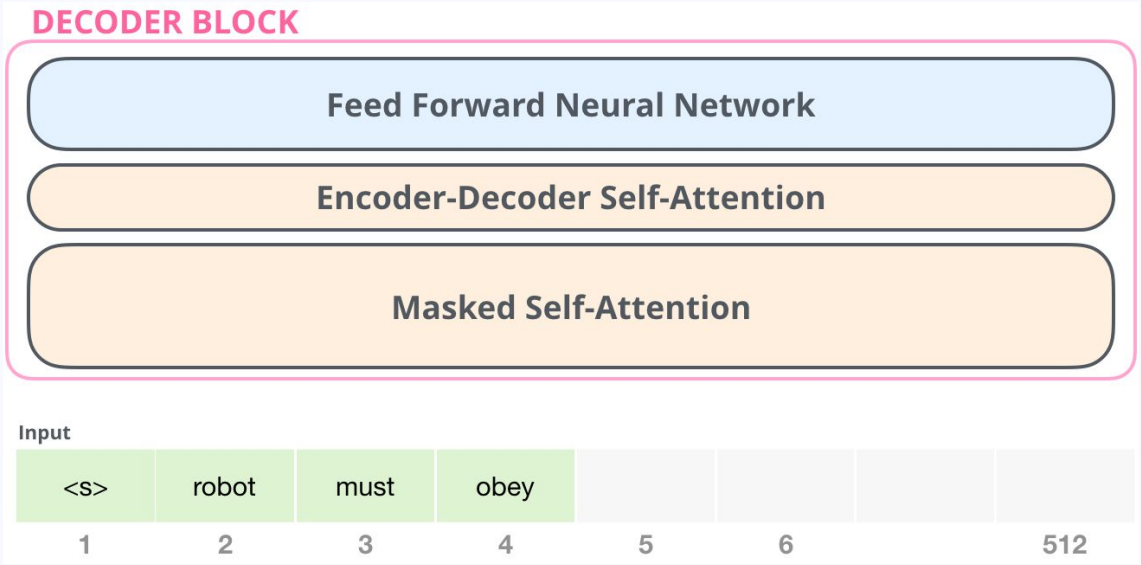
Each decoder processes the token, passes its output to the next decoder



https://jalammar.github.io/illustrated-gpt2/

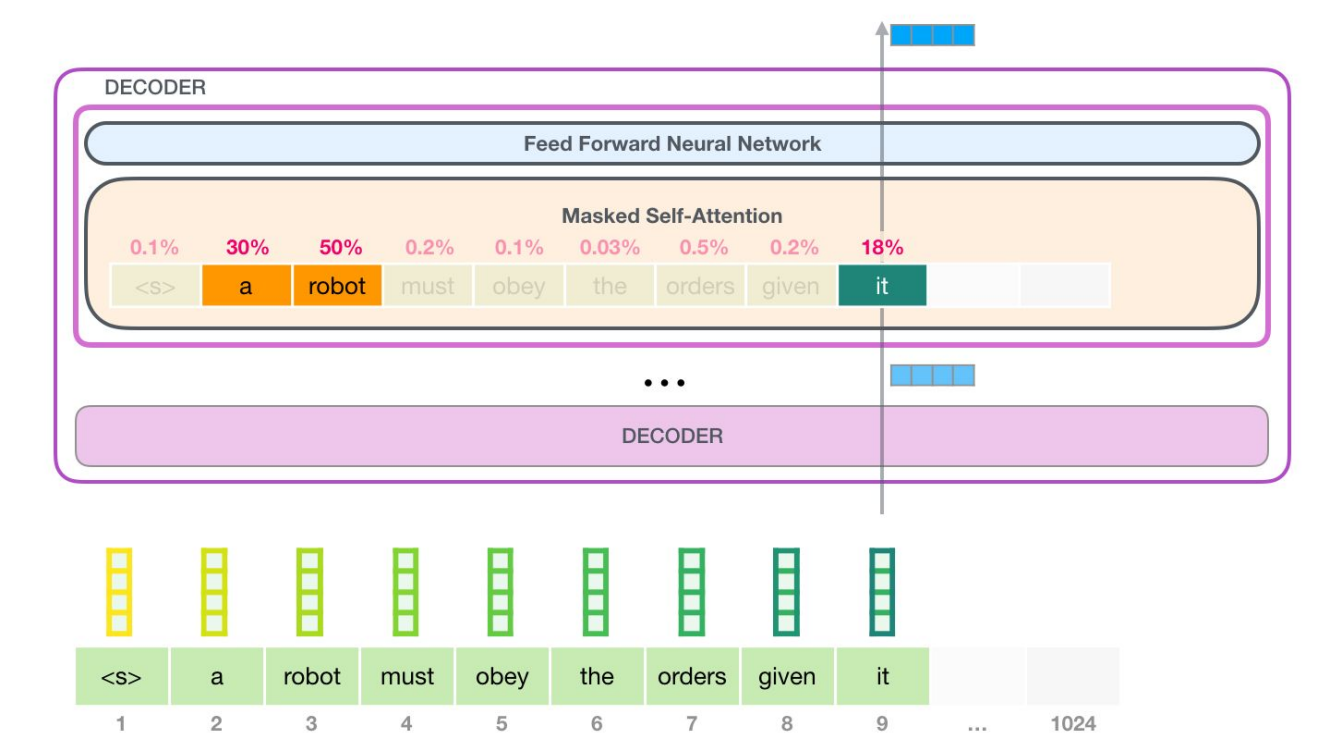
GPT Decoder block

Masked self-attention in the decoder block = access only to the output's previously predicted tokens!



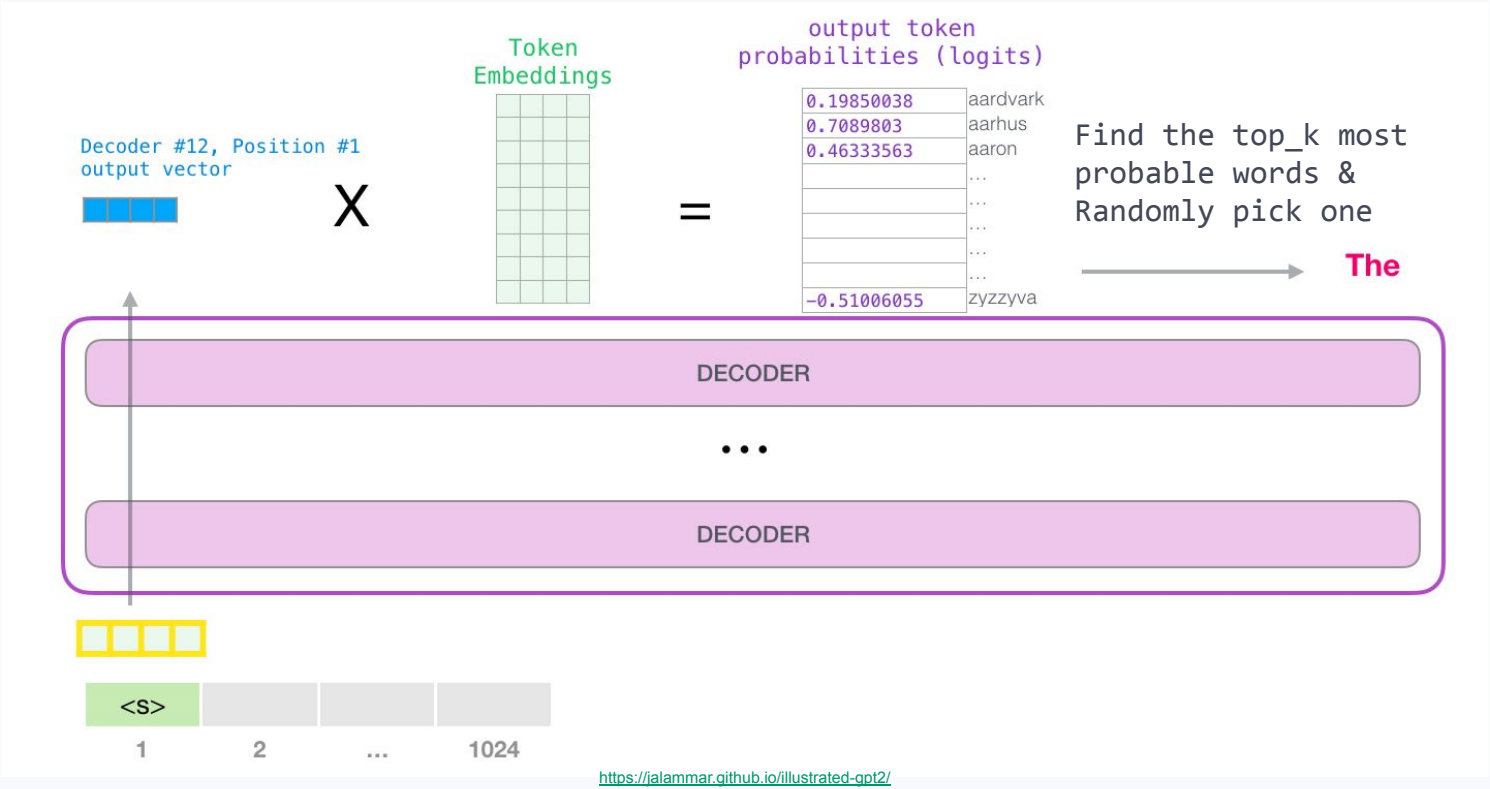
https://jalammar.github.io/illustrated-gpt2/

GPT Masked self-attention



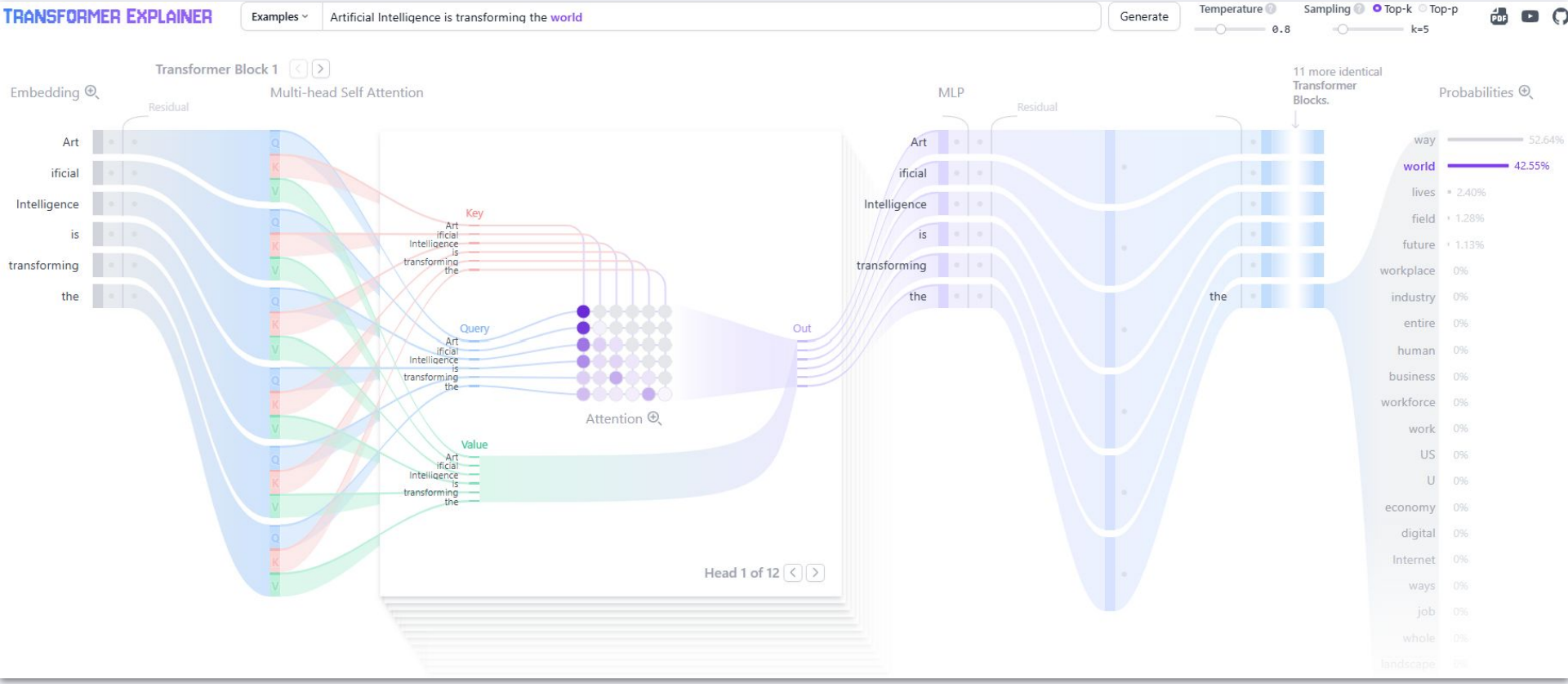
https://jalammar.github.io/illustrated-gpt2/

GPT Output

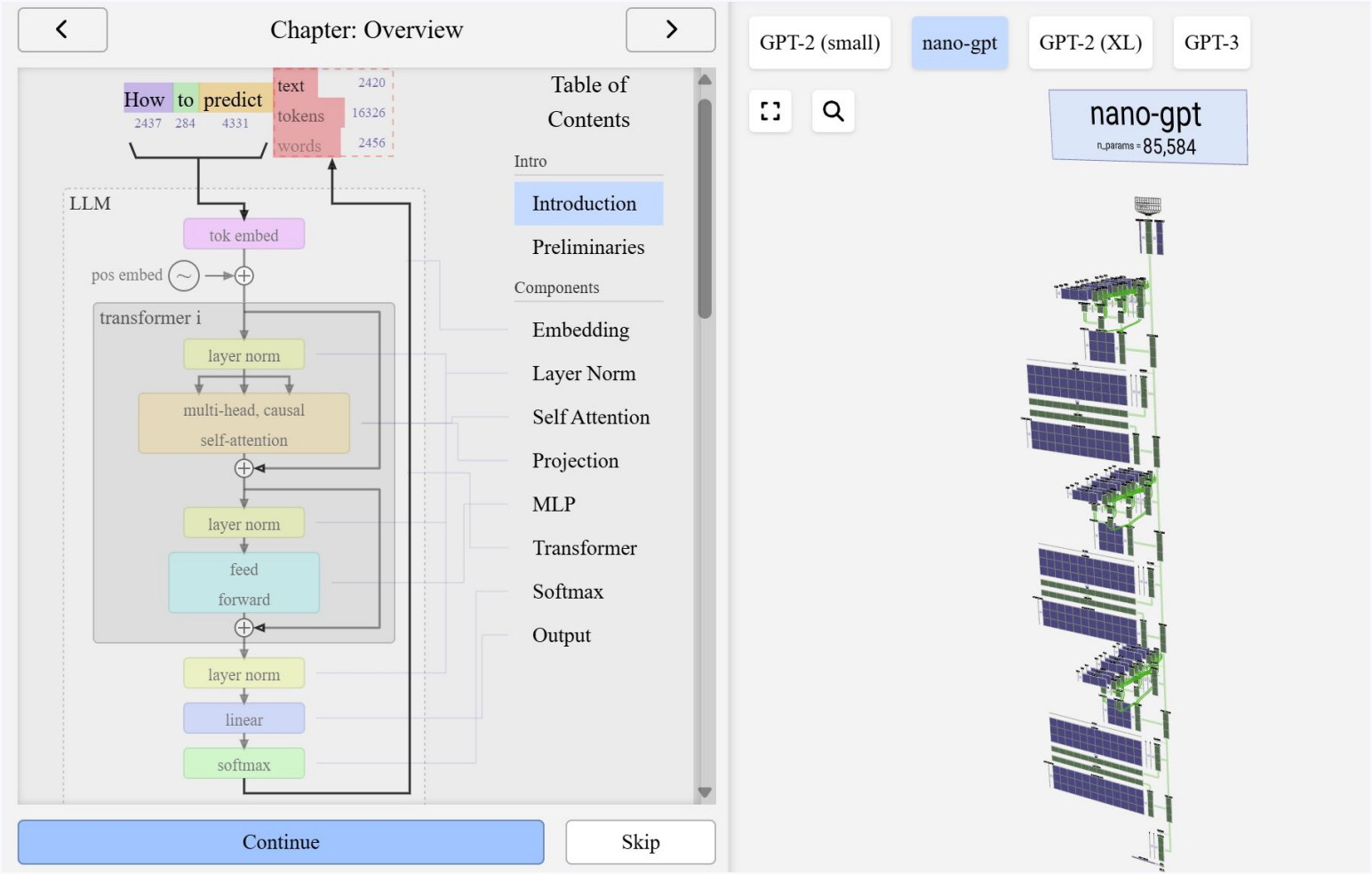


https://jalammar.github.io/illustrated-gpt2/

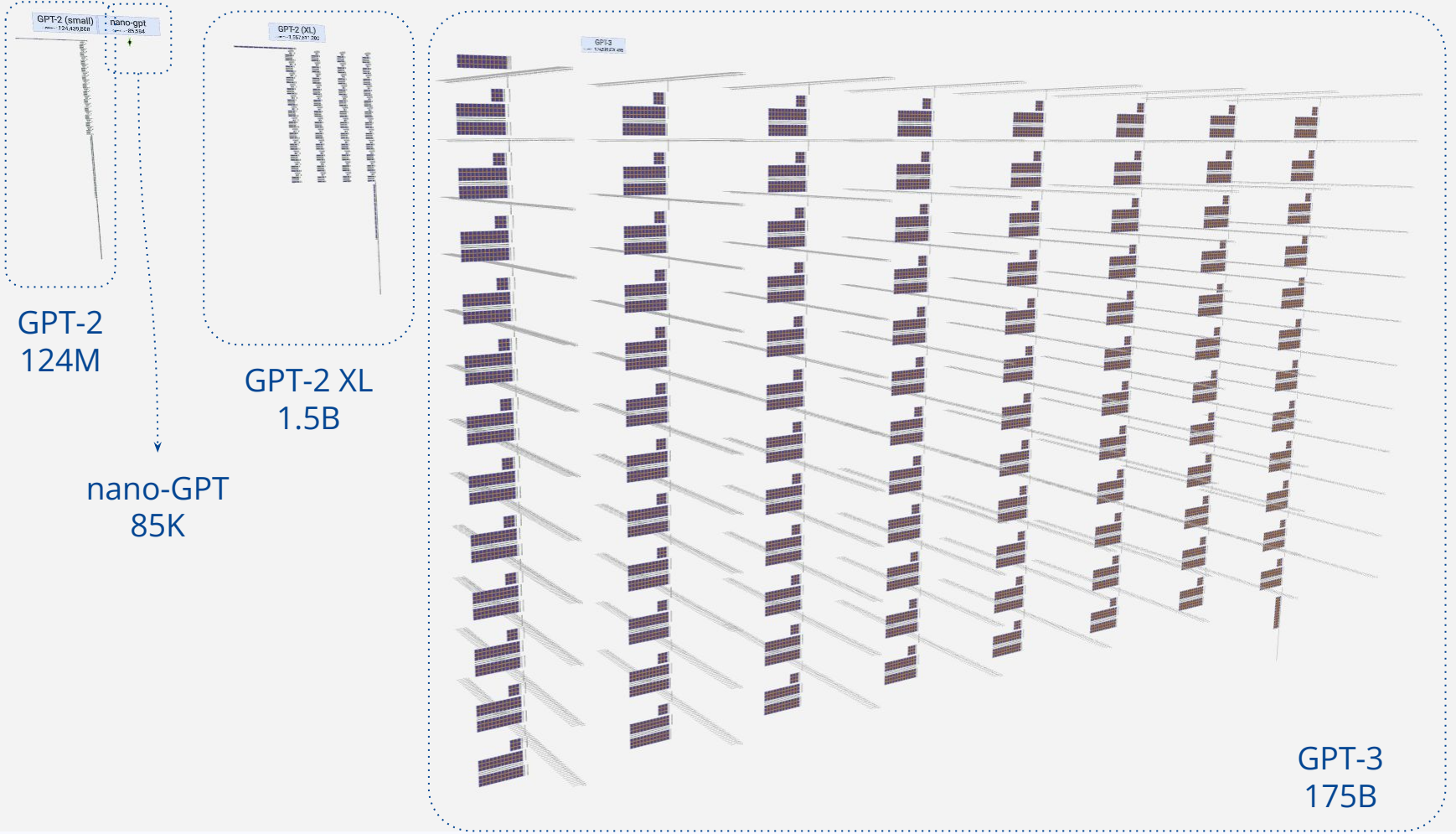
Transformer Explainer: Visualize how GPT-2 processes input & generates next token



https://poloclub.github.io/transformer-explainer/

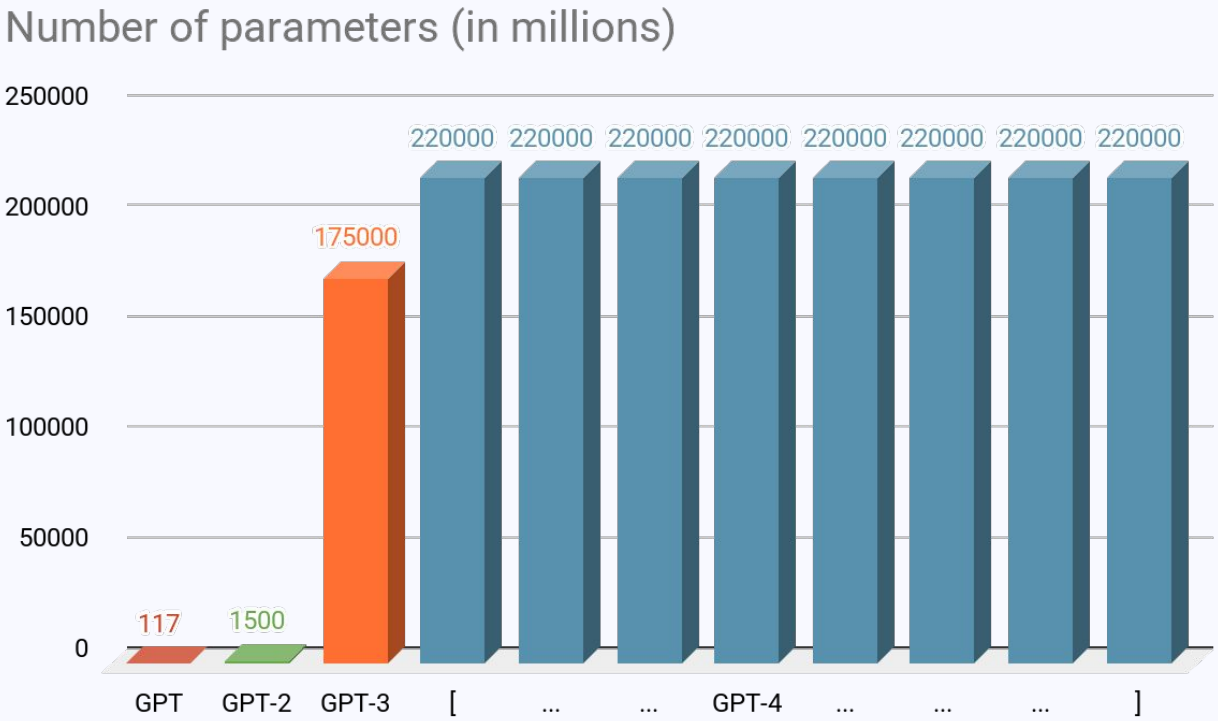


https://bbycroft.net/llm



https://bbycroft.net/llm

GPT Model Sizes



More LLMs

LLaMA models

- Also a **Transformer**-based architecture
- Bigger input embeddings (others: 512 dimensions, LLaMA: > 4.056 dims...)
- **Root Mean-Square normalization**: rescale vectors, not turn them ~0
- **Positional embeddings** in relation to the key-queries, not the token
- **Multi-query** self attention: same queries across attention heads

- LLaMA Instruct models: ideal for prompts that follow structured instructions

https://pub.towardsai.net/llama-explained-a70e71e706e9

Mistral models

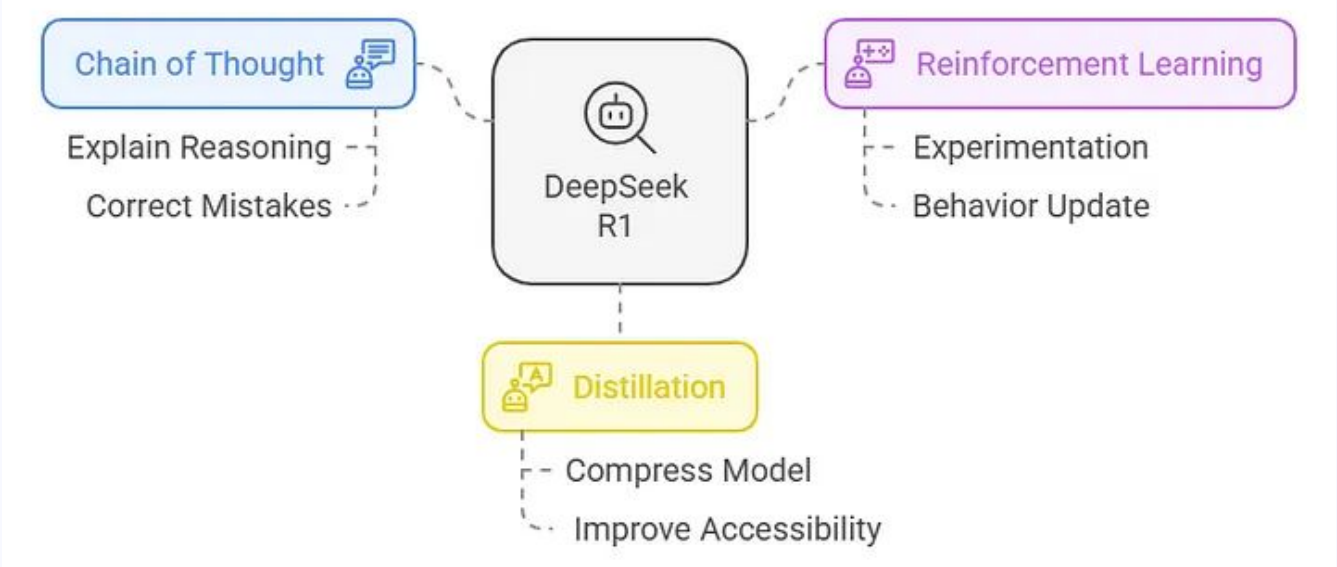
- Similar to LLaMa models
- **Optimizations** to improve memory use w.r.t. attention:

  Sliding Window attention, Rolling Buffer Cache, Pre-fill and chunking

https://medium.com/dair-ai/papers-explained-mistral-7b-b9632dedf580

DeepSeek

Employing strategies to improve upon existing Transformer architectures



Improving LLMs

- **Distillation**: transferring knowledge from a large model to a smaller one with an algorithm or with generated data (≠ pruning, ≠ compression)

- **Chain of Thought**: the model *has* to output the “thought” process of answer

- **Reinforcement Learning**: update the model with good answers to “forget” the bad ones
- **Supervised Finetuning**: finetune the model with high-quality outputs

https://medium.com/@tahirbalarabe2/deepseek-r1-explained-chain-of-thought-reinforcement-learning-and-model-distillation-0eb165d928c9

Practical Session

ANITI

Preprocessing input

Preprocessing input

```
from transformers import BertTokenizer

# Load the BERT tokenizer.
print('Loading BERT tokenizer...')
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased', do_lower_case=True)

# Print the original sentence.
print(' Original: ', sentences[0])

# Print the sentence split into tokens.
print('Tokenized: ', tokenizer.tokenize(sentences[0]))

# Print the sentence mapped to token ids.
print('Token IDs: ', tokenizer.convert_tokens_to_ids(tokenizer.tokenize(sentences[0])))
```

→ BERT's specific tokenizer, with functions, BERT vocabulary and special tokens

from pretrained = load the model from HF

preprocessing option

bert-base-uncased = type of BERT

→ tokenize function

→ encode in BERT vocabulary

```
Loading BERT tokenizer...
 Original:  One more pseudo generalization and I'm giving up.
Tokenized:  ['one', 'more', 'pseudo', 'general', '##ization', 'and', 'i', "'", 'm', 'giving', 'up', '.']
Token IDs:  [2028, 2062, 18404, 2236, 3989, 1998, 1045, 1005, 1049, 3228, 2039, 1012]
```

Preprocessing input

```
Loading BERT tokenizer...
 Original:  One more pseudo generalization and I'm giving up.
Tokenized:  ['one', 'more', 'pseudo', 'general', '##ization', 'and', 'i', "'", 'm', 'giving', 'up', '.']
Token IDs:  [2028, 2062, 18404, 2236, 3989, 1998, 1045, 1005, 1049, 3228, 2039, 1012]
```

long word: split in token + “function” subtoken!

…but what about the special tokens? Add them in the *encoding* stage!

```
input_ids = tokenizer.encode(sentences[1], add_special_tokens=True)
print('Token IDs: ',input_ids)
print('Tokenized: ',tokenizer.convert_ids_to_tokens(input_ids))
```

```
Token IDs:  [101, 2028, 2062, 18404, 2236, 3989, 1998, 1045, 1005, 1049, 3228, 2039, 1012, 102]
Tokenized:  ['[CLS]', 'one', 'more', 'pseudo', 'general', '##ization', 'and', 'i', "'", 'm', 'giving', 'up', '.', '[SEP]']
```

Preprocessing input

```
# `encode_plus` will:
#   (1) Tokenize the sentence.
#   (2) Prepend the `[CLS]` token to the start.
#   (3) Append the `[SEP]` token to the end.
#   (4) Map tokens to their IDs.
#   (5) Pad or truncate the sentence to `max_length`
#   (6) Create attention masks for [PAD] tokens.
encoded_dict = tokenizer.encode_plus(
                    sent,                      # Sentence to encode.
                    add_special_tokens = True, # Add '[CLS]' and '[SEP]'
                    max_length = 64,           # Pad & truncate all sentences.
                    pad_to_max_length = True,
                    return_attention_mask = True,   # Construct attn. masks.
                    return_tensors = 'pt',     # Return pytorch tensors.
               )
```

Preprocessing input

Attention mask: tokens to “show to/hide from” the model (for MLM)

Input		I	like	[MASK]	salad	.	
Tokens	[CLS]	i	like	[MASK]	salad	.	[SEP]
Encoded tokens	101	1045	2066	103	16521	1012	102
Attention mask	1	1	1	0	1	1	1

Finetuning

Finetuning

The process of adapting a pre-trained model for specific tasks or use cases

- Parameters of existing model + New data with specific topic/interest
- Either on the entire network or on some layers or in a new final layer

- **Goal**: specialize the model on a new task with few data, but also use the power of the model's learned representations
- Save up time and computational power!

Finetuning BERT for Sequence Classification

Let's head to Google Colab:

https://colab.research.google.com/drive/14EV6vfTECPxq_xG9xajAxvd6tBmiGLFf?usp=sharing

We will use the PyTorch and Hugging Face libraries!

Finetuning

with PyTorch

```
# For each epoch...
for epoch_i in range(0, epochs):

    # ========================================
    #               Training
    # ========================================

    # Perform one full pass over the training set.

    print("")
    print('======== Epoch {:} / {:} ========'.format(epoch_i + 1, epochs))
    print('Training...')

    # Measure how long the training epoch takes.
    t0 = time.time()

    # Reset the total loss for this epoch.
    total_train_loss = 0

    # Put the model into training mode. Don't be mislead--the call to
    # `train` just changes the *mode*, it doesn't *perform* the training.
    # `dropout` and `batchnorm` layers behave differently during training
    # vs. test (source: https://stackoverflow.com/questions/51433378/what-d
    model.train()
```

with HuggingFace

```
training_args = TrainingArguments(
    output_dir="my_awesome_model",
    learning_rate=2e-5,
    per_device_train_batch_size=16,
    per_device_eval_batch_size=16,
    num_train_epochs=2,
    weight_decay=0.01,
    evaluation_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    push_to_hub=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_imdb["train"],
    eval_dataset=tokenized_imdb["test"],
    tokenizer=tokenizer,
    data_collator=data_collator,
    compute_metrics=compute_metrics,
)

trainer.train()
```

Language Generation

How to generate the best text?

- **Greedy search**: always return the top probability
- **Beam search**: “remember” other top probabilities, return the top probability
- **Sampling**: allow our model to pick words randomly… or randomly from the top ones!

Let’s test them all in the Google Colab!

https://colab.research.google.com/drive/14EV6vfTECPxq_xG9xajAxvd6tBmiGLFf?usp=sharing

Thank you for your (self) attention!

Homework: **New Notebook!**

ANITI

Terminology cheatsheet

- **Temperature**: control the smoothness of the probability distribution (0-1)

  🧊 = pick token with highest prob. / 🔥 = make token probs more uniform

- **Hyperparameters**: NOT the model's parameters (weights & biases)!

  Variables that control the model's training and we choose, e.g.:

  number of layers, size of input embeddings,

Further reading

- Reading:
  - The Illustrated Transformer – Jay Alammar
  - The Illustrated BERT, ELMo, and co.– Jay Alammar
  - The Illustrated GPT-2– Jay Alammar
  - Illustrated Guide to Transformers- Step by Step Explanation - Michael Phi

- Video Courses:
  - Jay Alammar's course: How transformer LLMs work
  - StatsQuest

- Notebooks:
  - Tensor2Tensor intro: Colab Notebook with visualizations
  - More notebooks folder: look at embeddings, run BERT, prompt LLMs!